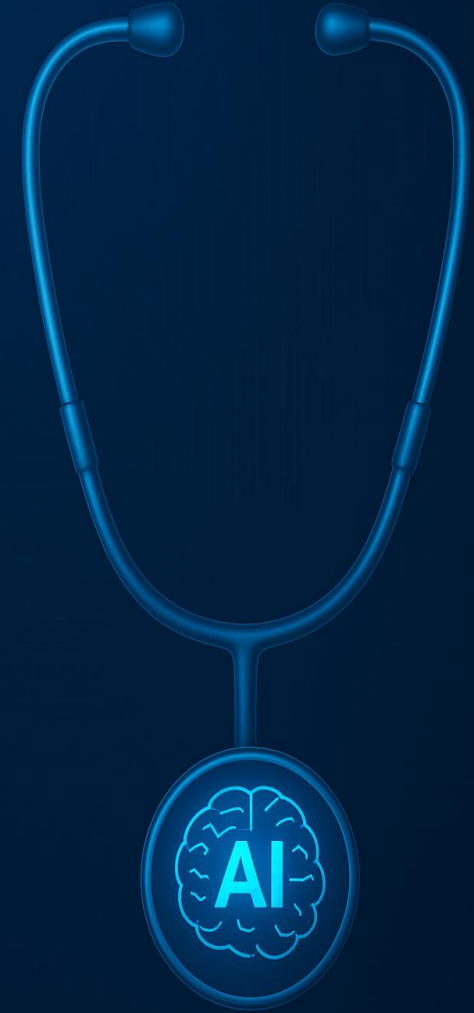


Cours par Mohamad Forouzanfar

L'IA DANS LES SOINS DE SANTÉ

GTS880

Sujets spéciaux en technologies de la santé



Cours 5

PRÉVISION D'ÉVÉNEMENTS DE SANTÉ À L'AIDE DE L'ANALYSE DE SÉRIES TEMPORELLES

Rappel

LA DATE LIMITE POUR CHOISIR L'ARTICLE DU PROJET FINAL EST LE **VENDREDI 10 OCTOBRE**.

L'EXAMEN **INTRA 1** AURA LIEU LE **22 OCTOBRE**

Objectifs

- Formuler notre **prévision (forecasting)**.
- Apprendre les **techniques statistiques de modélisation de séries temporelles**. (Base essentielle pour comprendre les tendances des données temporelles et faire des prédictions de référence.)
- Etudier les **Réseaux Neuronaux Récurrents (RNN)** pour les séries temporelles non stationnaires, y compris les signaux physiologiques. (Crucial pour capturer des motifs complexes et évolutifs que les méthodes traditionnelles manquent souvent.)
- Apprendre le fonctionnement des **Transformers**. (De plus en plus puissants pour les **dépendances à plus long terme** dans les séries temporelles, offrant des avantages par rapport aux RNN dans certains cas.)

Qu'est-ce qu'une série temporelle ?

Les séries temporelles sont équivalentes à des signaux indexés par le temps, contrairement à, par exemple, l'espace (telles que les images).

La prévision (forecasting) de séries temporelles implique de prédire les valeurs futures de la série temporelle et/ou d'une variable dépendante associée en utilisant les informations disponibles.

Les informations disponibles comprennent :

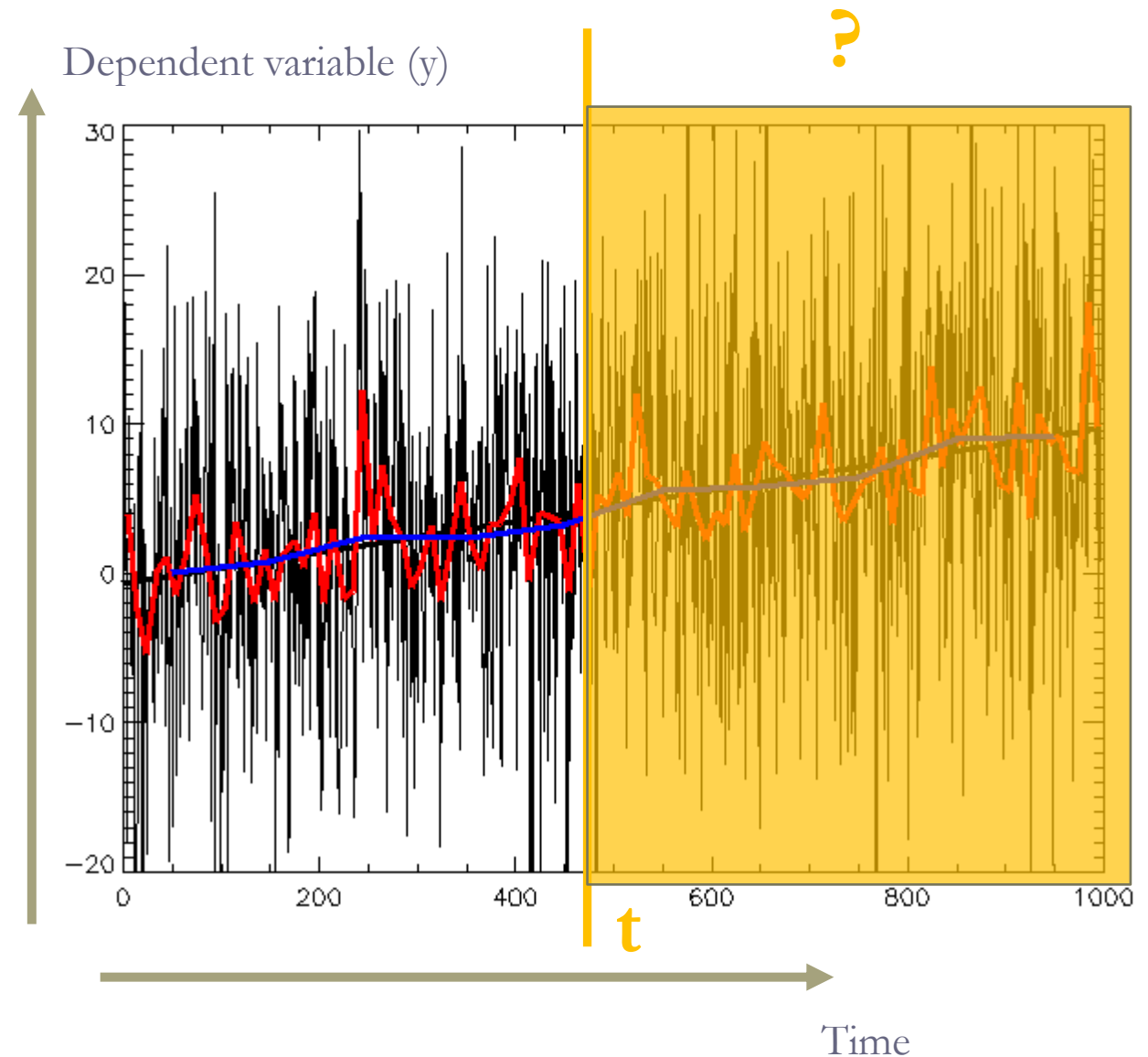
- les **variables endogènes** (les valeurs actuelles et passées de la série temporelle elle-même) et
- les **variables exogènes** (d'autres séries de données pertinentes).

À un instant donné t , nous pouvons écrire le problème de prévision comme suit :

$$\hat{y}_{t+H} = f(\text{informations actuelles et passées})$$

- $\hat{y}_{t+H}$: Valeur Prédite. La valeur que l'on cherche à estimer pour l'horizon futur $t+H$.
- $f(\dots)$: Fonction de Prévision (ou Modèle). La relation mathématique (algorithme).

Exemple : En utilisant les 12 dernières heures de mesures de la pression artérielle (endogène) et d'autres signaux physiologiques les antécédents du patient (exogènes), on peut prédire sa valeur future ou anticiper une crise cardiaque.



Prévision des séries temporelles

$$\hat{y}_{t+H} = f(\text{informations actuelles et passées})$$

Horizon

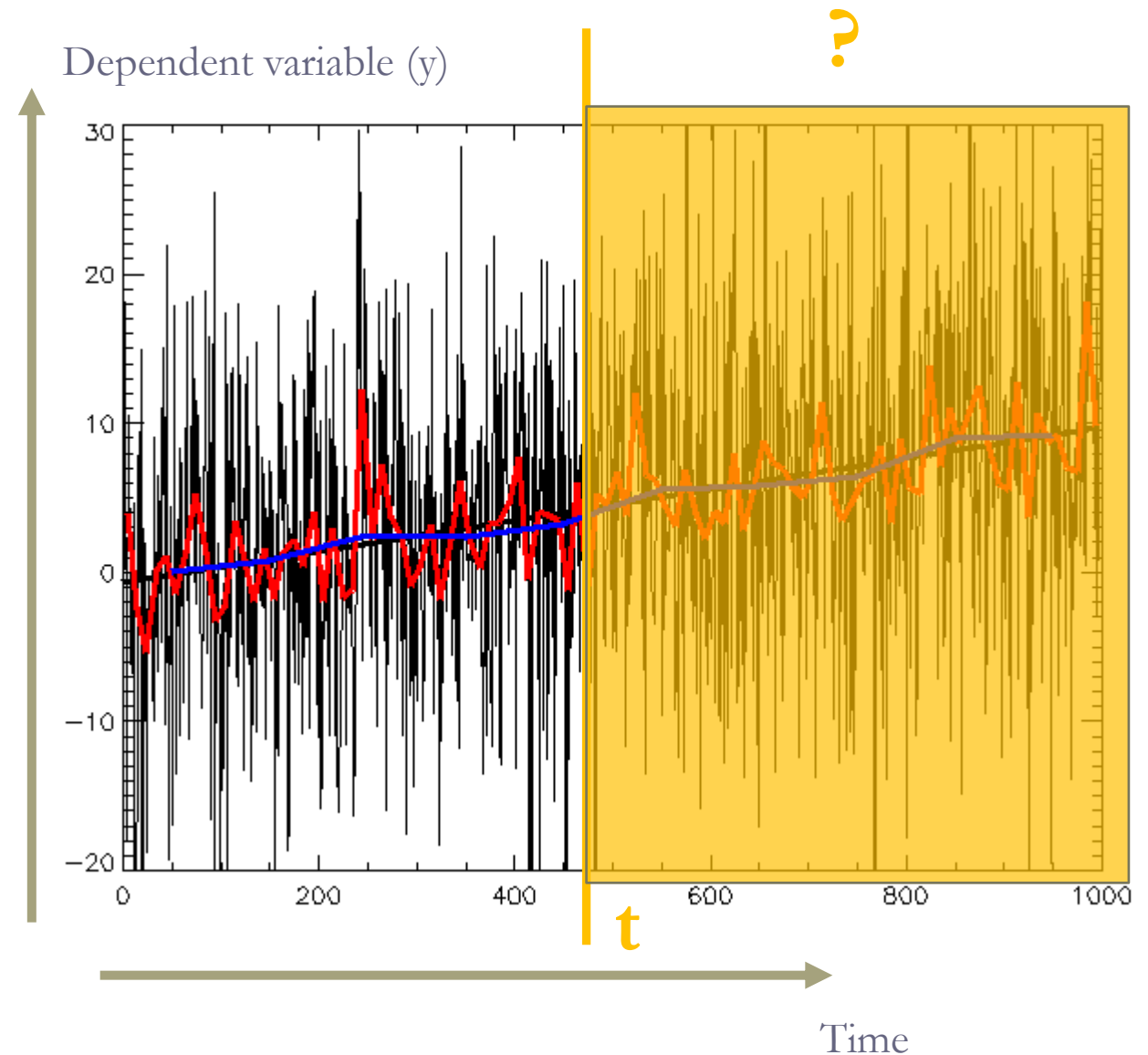
Modèle

où H est l'horizon temporel futur auquel la prévision est effectuée. f est le modèle (fonction) qui relie les informations actuelles et passées disponibles à la valeur de y dans le futur.

Le modèle peut être mathématique/statistique ou un modèle d'apprentissage automatique.

$$\hat{y}_{t+H} = f(y_t, y_{t-1}, y_{t-2}, \dots, y_1, x_t, x_{t-1}, x_{t-2}, \dots, x_1)$$

où y représente la série temporelle (**endogène**) et x représente une série temporelle **exogène**.



Les taux de glucose sanguin (glycémie) sont régulés par la division autonome du système nerveux qui module la libération de deux hormones : l'insuline et le glucagon, par le pancréas.

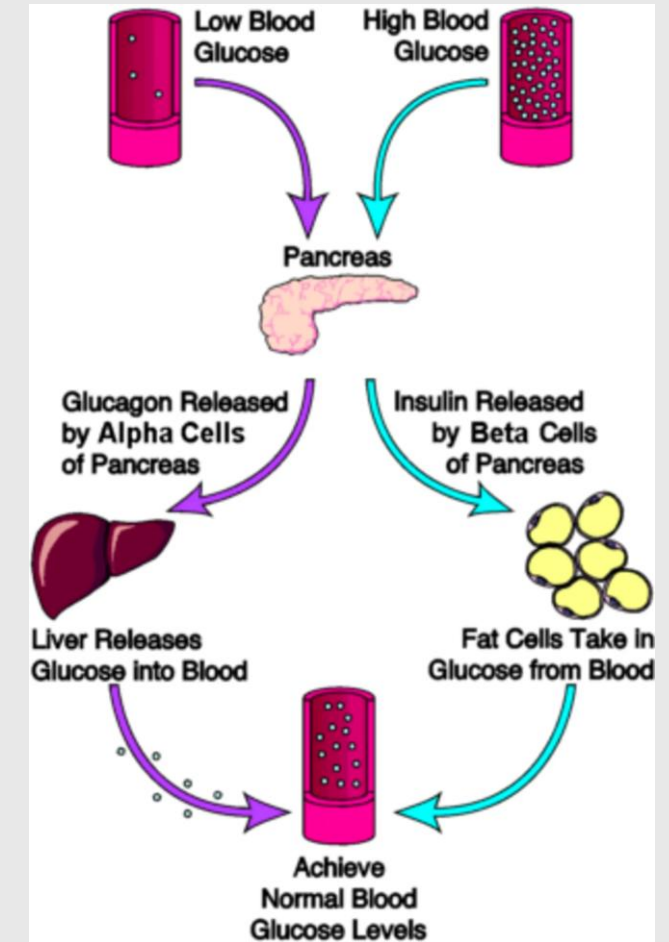
- Le glucagon augmente votre taux de sucre dans le sang et l'empêche de baisser trop bas,
- tandis que l'insuline diminue les taux de sucre dans le sang.

Lorsque vous avez le diabète, vous pouvez avoir une glycémie élevée (hyperglycémie) ou une glycémie basse (hypoglycémie), ce qui nécessite une surveillance de la glycémie.

Le diabète est une maladie dans laquelle votre corps ne peut pas produire d'insuline ou ne peut pas utiliser correctement l'insuline qu'il produit.

Une glycémie à jeun de 99 mg/dL (milligrammes par décilitre) ou moins est normale, 100 à 125 mg/dL indique un prédiabète, et 126 mg/dL ou plus indique le diabète. L'hypoglycémie survient lorsque votre glycémie est inférieure à 70 mg/dL.

Exemple : Taux de glucose dans le sang (BGL)



Régulation du taux de glucose dans le sang

<https://www.endocrineweb.com/conditions/diabetes/normal-regulation-blood-glucose>

@ 7

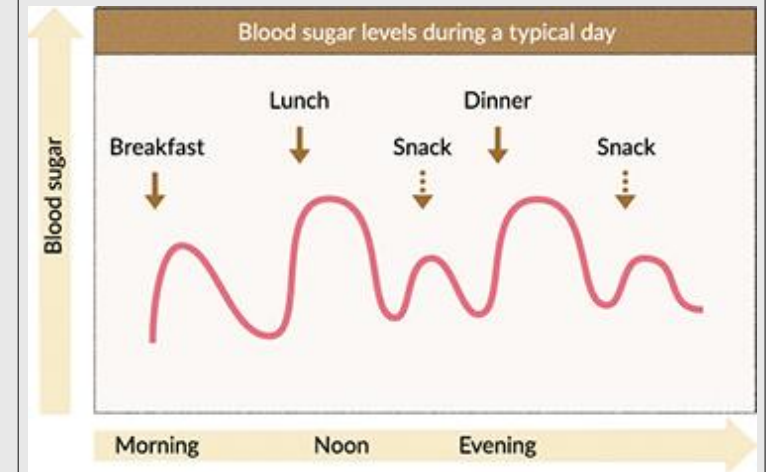
Exercice 1 : Identification des Variables dans la Prédiction du Taux de Glucose

Considérez le problème de la prédiction du taux de glucose sanguin (glycémie) chez un individu. Dans ce contexte, identifiez clairement :

1- Les variables endogènes : Quelles sont les variables dont la valeur est déterminée ou expliquée par le modèle de prédiction lui-même ?

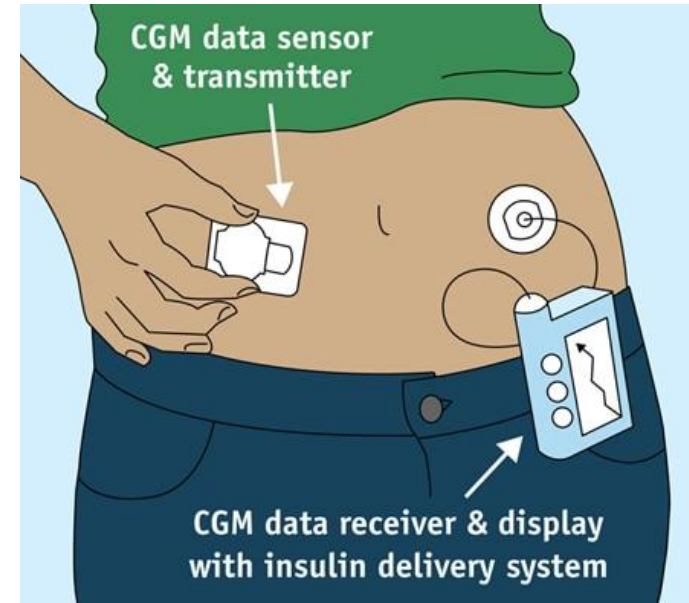
2- Les variables exogènes : Quelles sont les variables dont la valeur est déterminée en dehors du modèle de prédiction, mais qui peuvent influencer le taux de glucose sanguin ? Donnez des exemples spécifiques de ces variables.

Exemple : Taux de glucose dans le sang (BGL) (glycémie)

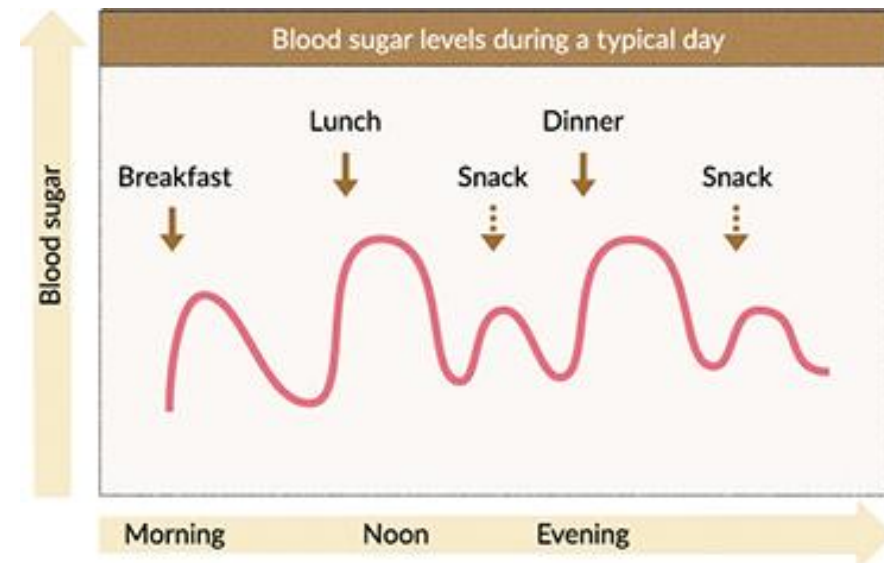


Série Temporelle BGL

La glycémie (taux de glucose sanguin) peut être surveillée en continu par un petit capteur sous-cutané, généralement sur le ventre ou le bras. Le capteur mesure le glucose interstitiel et le teste fréquemment. Un transmetteur envoie ces données sans fil pour la surveillance.



<https://www.niddk.nih.gov/health-information/diabetes/overview/managing-diabetes/continuous-glucose-monitoring>



<https://www.otsuka.co.jp/en/health-and-illness/glycemic-index/glucose-level/>

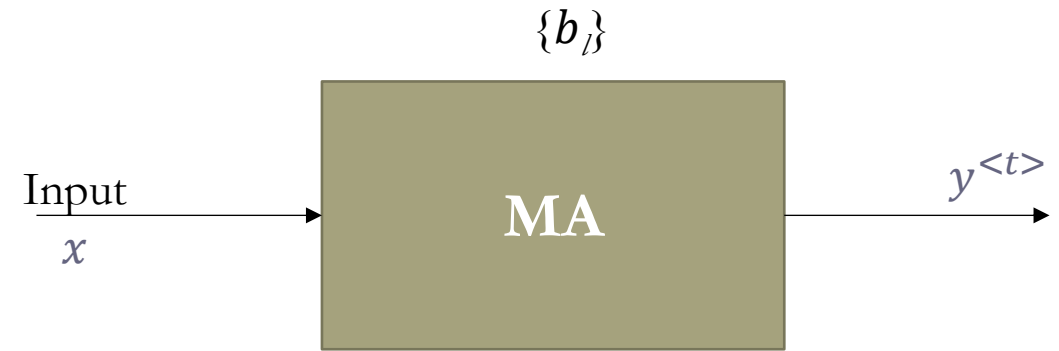
Modèles Stochastique de Base

Moyenne mobile –Moving Average: MA(q)

La moyenne mobile fournit un modèle paramétrique simple permettant de prédire la sortie des processus physiologiques comme une combinaison linéaire des valeurs actuelles et passées des entrées :

$$y^{<t>} = \sum_{l=1}^q b_l x^{<t-l>} + e^{<t>}$$

- $y^{<t>}$ est la valeur de la série temporelle au moment t
- $x^{<t>}$ est la valeur de la séquence d'entrée (série exogène) au moment t
- b_l est le poids (coefficients ou paramètres) du modèle (filtre)
- $e^{<t>}$ est un terme d'erreur qui représente la partie inexpliquée de y par x
- q est l'ordre du modèle



Représentation d'une série temporelle comme la sortie d'un filtre à moyenne mobile linéaire

Modèles Stochastiques de Base

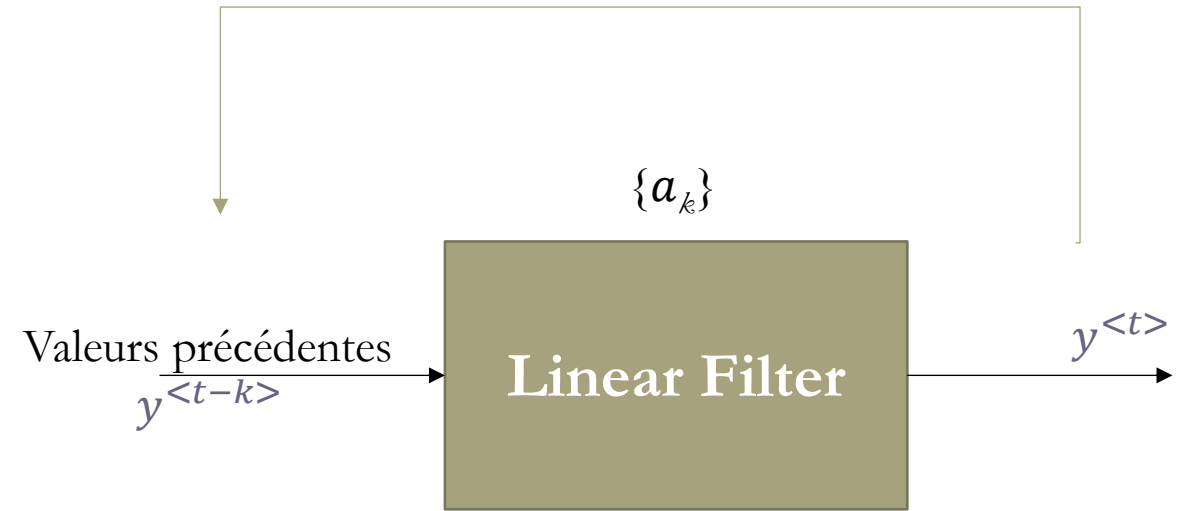
Modèle autorégressif – AR(p)

Un modèle stochastique qui peut être extrêmement utile dans la représentation de certaines séries rencontrées en pratique est le modèle dit autorégressif (AR).

Dans un modèle AR, la valeur actuelle du processus est exprimée comme une combinaison linéaire finie des valeurs précédentes du processus. Un processus AR d'ordre p peut être exprimé comme suit :

$$y^{<t>} = - \sum_{k=1}^p a_k y^{<t-k>} + e^{<t>}$$

- $y^{<t>}$ est la valeur de la série temporelle au moment t
- a_k est le poids (coefficients ou paramètres) du modèle (filtre)
- $e^{<t>}$ est un terme d'erreur qui représente la partie inexpliquée de $y^{<t>}$ par ses valeurs précédentes.



Représentation d'un modèle AR sous la forme d'un filtre linéaire

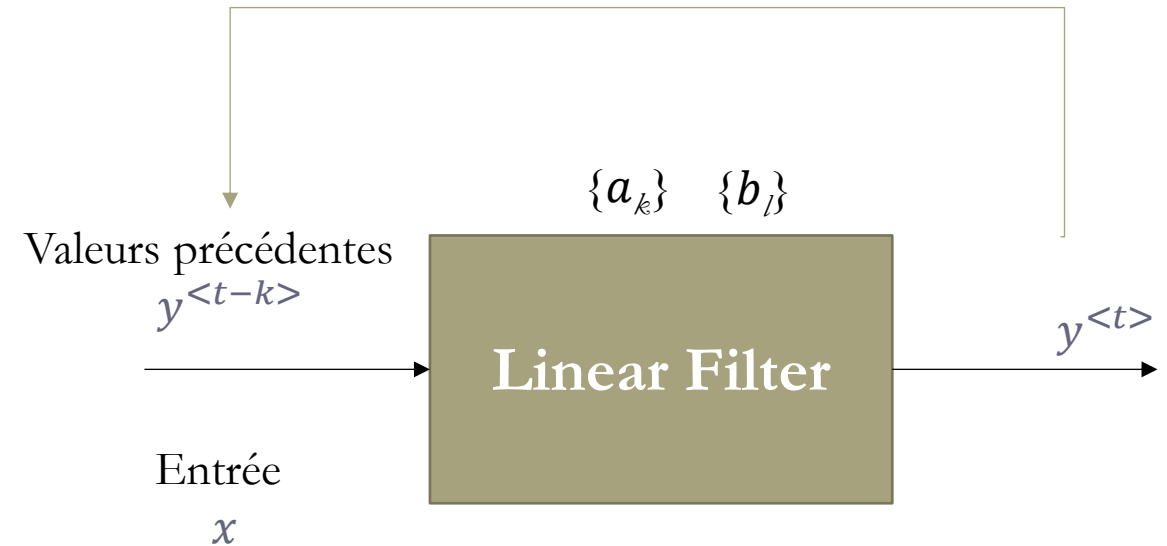
Modèles Stochastiques de Base

Modèles autorégressifs à moyenne mobile mixtes – ARMA(p, q)

Afin d'obtenir une plus grande flexibilité dans l'ajustement des séries temporelles, il peut être avantageux d'inclure à la fois des termes autorégressifs et des termes de moyenne mobile dans le modèle. Ceci conduit au modèle autorégressif à moyenne mobile mixte :

$$y^{<t>} = \sum_{l=1}^q b_l x^{<t-l>} - \sum_{k=1}^p a_k y^{<t-k>}$$

Comment estimer les paramètres du modèle ?



Représentation d'un processus ARMA

Prévision de la glycémie (BGL)

Le tableau de droite présente une liste de variables que nous avons enregistrées avec une résolution temporelle de 5 minutes, ainsi que le BGL.

Comment concevriez-vous un modèle ARMA pour prédire le BGL en utilisant ces informations supplémentaires ?

Variable name	Description	Value/units in given time period ^a
Calories consumed	From food and beverages	Total calories
Meds taken	Of any type	0 = no, 1 = yes
Insulin taken	Variety of slow, medium, and fast acting insulin medications	0 = no, 1 = yes
Movement intensity	Vector magnitude, square root of the sum of the squared 3-axis acceleration readings	Average vector magnitude ^b
Exercise time	Aerobic, weights, walking, and so forth	Total minutes
Trip time	Time spent travelling by any mode	Total minutes
Automobile time	Time spent travelling by personal automobile	Total minutes
Public transit time	Time spent travelling by public transit (bus, streetcar, subway, and train)	Total minutes
At-home time	Time spent at home	Total minutes
Work/school time	Time spent at work or school	Total minutes
Shopping time	Time spent shopping (grocery, clothing, etc.)	Total minutes
Restaurants time	Time spent in a restaurant of any type	Total minutes
Out-of-home time	Time spent outside the home	Total minutes
Time spent with others	Time spent with other people involved	Total minutes
Time of day	Daytime (7:00AM to 7:00PM) or nighttime	0 = daytime, 1 = nighttime
Distance home	Straight-line distance between current location and home location	Average distance in kilometers

@Doherty, S. T., & Greaves, S. P. (2015). Time-series analysis of continuously monitored blood glucose: the impacts of geographic and daily lifestyle factors. Journal of diabetes research, 2015.

Prévision de la glycémie (BGL)

Une solution consiste à utiliser les valeurs précédentes de BGL dans un modèle AR, et considérer d'autres variables comme des séries exogène (à l'aide des modèles MA) :

$$y^{<t>} = - \sum_{k=1}^p a_k y^{<t-k>} + \sum_{m=1}^r b_{1,m} x^{1,<t-m>} + \sum_{n=1}^s b_{2,n} x^{2,<t-n>} + \dots$$

Pour prévoir l'avenir, au lieu de $y^{<t>}$, je peux utiliser $y^{<t+1>}$ ou $y^{<t+H>}$ sur le côté gauche de l'équation.

Variable name	Description	Value/units in given time period ^a
Calories consumed	From food and beverages	Total calories
Meds taken	Of any type	0 = no, 1 = yes
Insulin taken	Variety of slow, medium, and fast acting insulin medications	0 = no, 1 = yes
Movement intensity	Vector magnitude, square root of the sum of the squared 3-axis acceleration readings	Average vector magnitude ^b
Exercise time	Aerobic, weights, walking, and so forth	Total minutes
Trip time	Time spent travelling by any mode	Total minutes
Automobile time	Time spent travelling by personal automobile	Total minutes
Public transit time	Time spent travelling by public transit (bus, streetcar, subway, and train)	Total minutes
At-home time	Time spent at home	Total minutes
Work/school time	Time spent at work or school	Total minutes
Shopping time	Time spent shopping (grocery, clothing, etc.)	Total minutes
Restaurants time	Time spent in a restaurant of any type	Total minutes
Out-of-home time	Time spent outside the home	Total minutes
Time spent with others	Time spent with other people involved	Total minutes
Time of day	Daytime (7:00AM to 7:00PM) or nighttime	0 = daytime, 1 = nighttime
Distance home	Straight-line distance between current location and home location	Average distance in kilometers

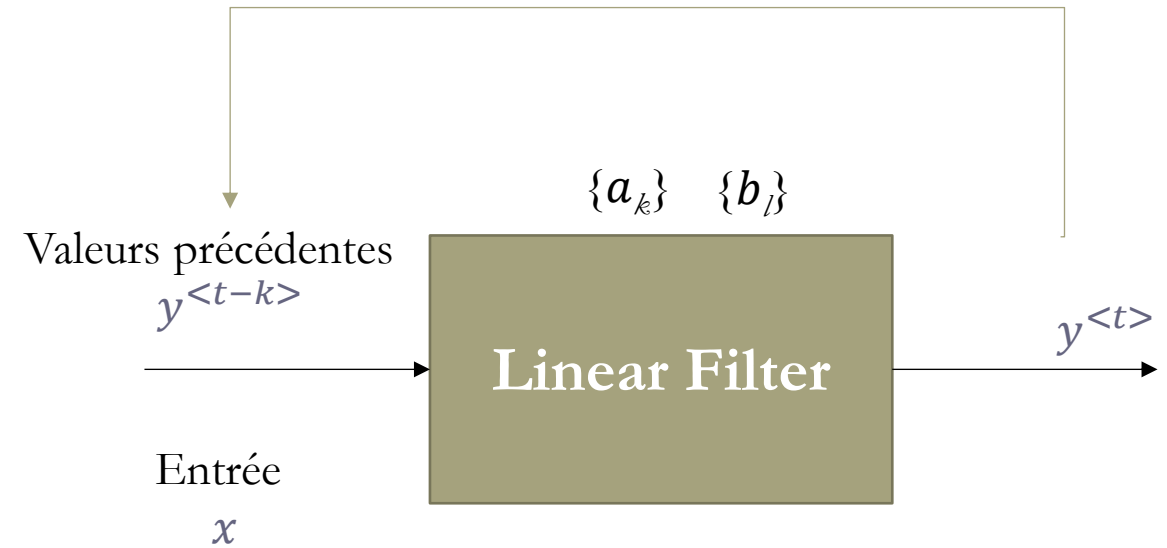
@Doherty, S. T., & Greaves, S. P. (2015). Time-series analysis of continuously monitored blood glucose: the impacts of geographic and daily lifestyle factors. Journal of diabetes research, 2015.

Limitations des modèles MA et AR

Les modèles AR et MA supposent que la série temporelle est **stationnaire**, ce qui signifie que :

La moyenne, la variance et les relations entre les valeurs restent constantes au cours du temps.

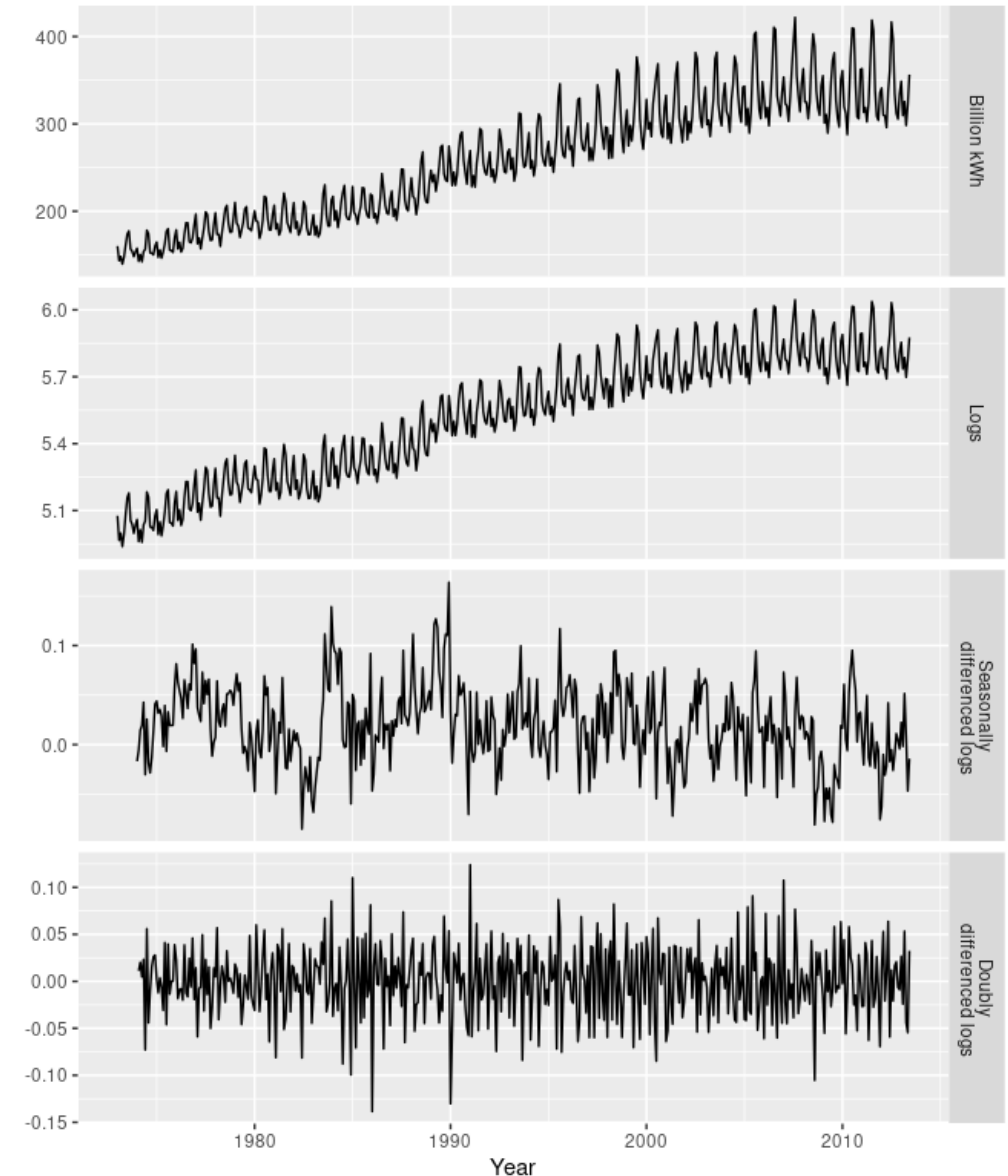
Mais dans les données du monde réel (comme les signaux physiologiques), les tendances et des changements brusques rendent souvent les données non stationnaires.



Représentation d'un processus ARMA

Transformer une série pour la rendre stationnaire

Il existe des solutions simples pour stabiliser la série, comme **différencier** le signal (prendre la dérivée) pour supprimer la tendance, ou utiliser le **logarithme** pour réduire les variations d'échelle. Cependant, ces méthodes ne suffisent pas toujours et ne sont pas universellement applicables.



Réseau neuronal récurrent (RNN)

Au-delà des limites de stationnarité

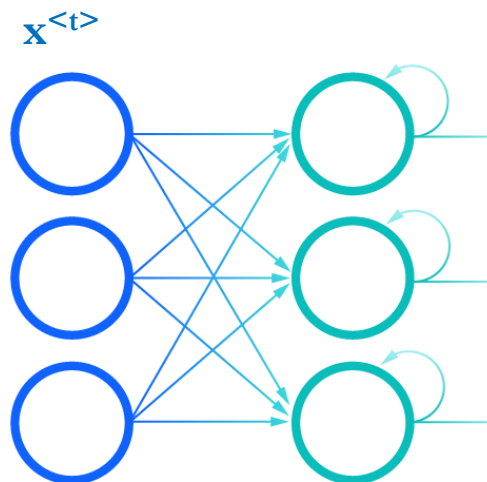
Les réseaux de neurones récurrents (RNN) sont une variante des réseaux de neurones conçue pour traiter des données séquentielles en intégrant (ou conservant) le contexte des informations passées.

Contrairement aux réseaux classiques à propagation avant (MLP), où les connexions sont unidirectionnelles de l'entrée vers la sortie, les RNN entièrement connectés possèdent des connexions récurrentes entre tous les neurones.

Grâce à leurs connexions de rétroaction, les RNN créent une mémoire interne (état caché).

Cette mémoire leur permet d'**apprendre et de suivre les dépendances temporelles** dans les données, ce qui les rend particulièrement adaptés pour modéliser des caractéristiques **non linéaires** et **non stationnaires** (comme les tendances ou les changements brusques) et fournir des prédictions plus précises.

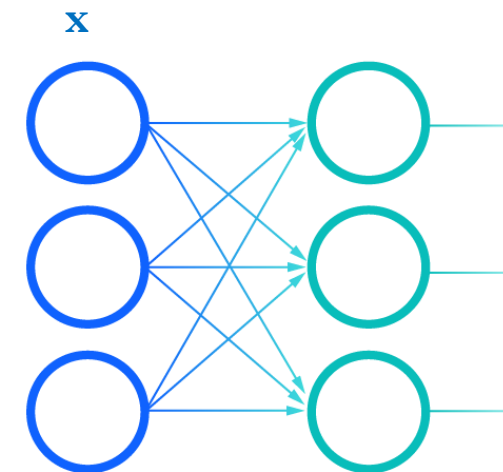
RNN



$$\mathbf{x}^{<t>} = \begin{bmatrix} \mathbf{x}^{<t>}(1) \\ \mathbf{x}^{<t>}(2) \\ \vdots \\ \mathbf{x}^{<t>}(d) \end{bmatrix}$$

$t = 1 \text{ à } T$

MLP



$$\mathbf{x} = \begin{bmatrix} x(1) \\ x(2) \\ \vdots \\ x(d) \end{bmatrix}$$

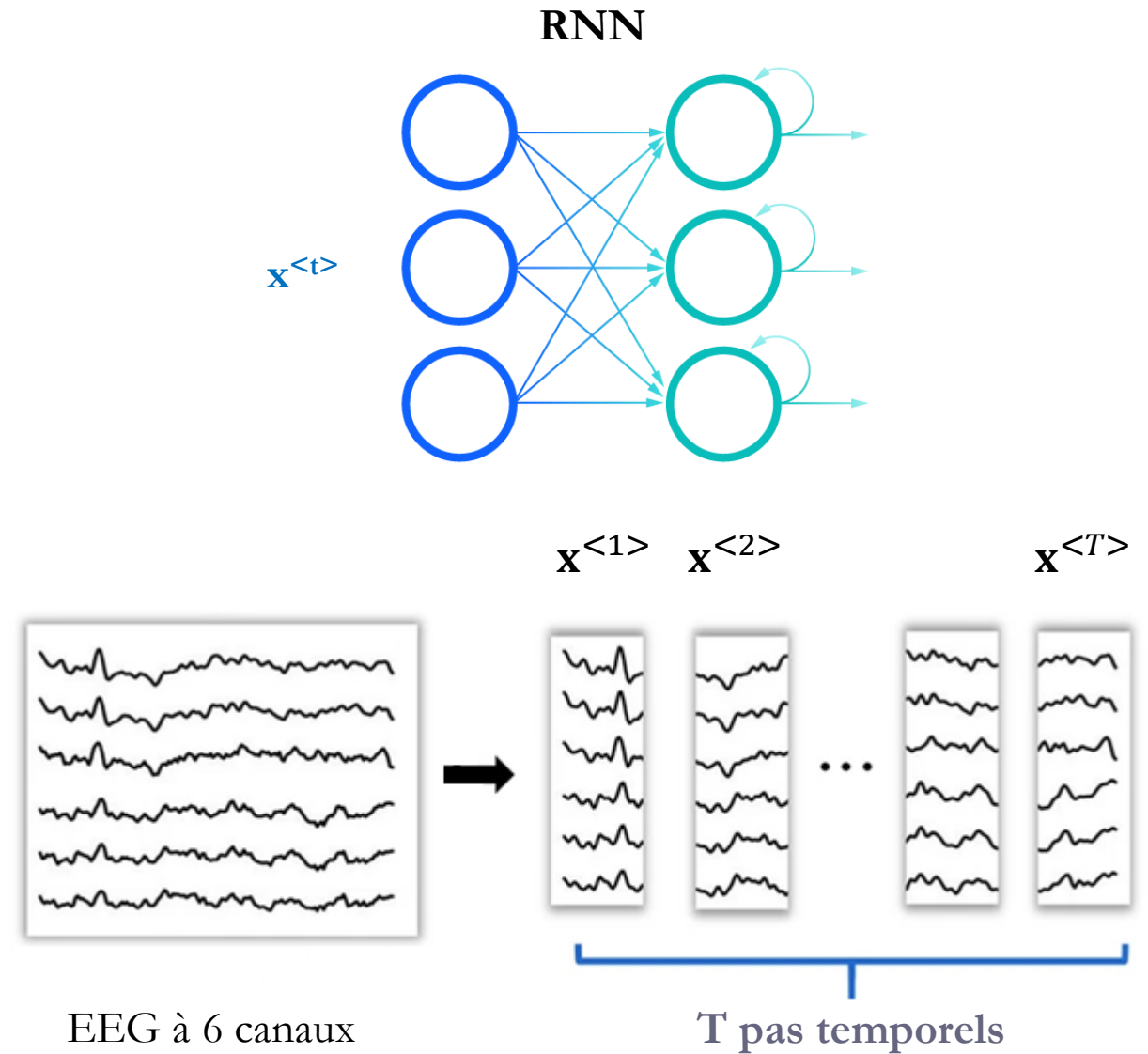
Comparaison des réseaux neuronaux récurrents (à gauche) et des réseaux neuronaux à propagation avant (à droite)

<https://www.ibm.com/cloud/learn/recurrent-neural-networks>

Réseau neuronal récurrent (RNN)

Au-delà des limites de stationnarité

Exemple de traitement d'un signal EEG à 6 canaux avec un RNN. Le signal est divisé en T intervalles de temps.



Réseau neuronal à propagation avant (MLP)

Dans un perceptron multicouche (MLP), **la transformation de une couche cachée (à propagation avant) peut être représentée sous forme matricielle**. Par exemple, la sortie du neurone i dans la première couche cachée peut s'écrire :

$$y(i) = g(w_{x1,i}x(1) + w_{x2,i}x(2) + \dots + w_{xd,i}x(d) + b_i) = \mathbf{w}_i^{\text{tr}} \mathbf{x} + b_i$$

Ensuite, la sortie d'une couche cachée comportant r neurones peut être exprimée sous forme vectorielle comme suit :

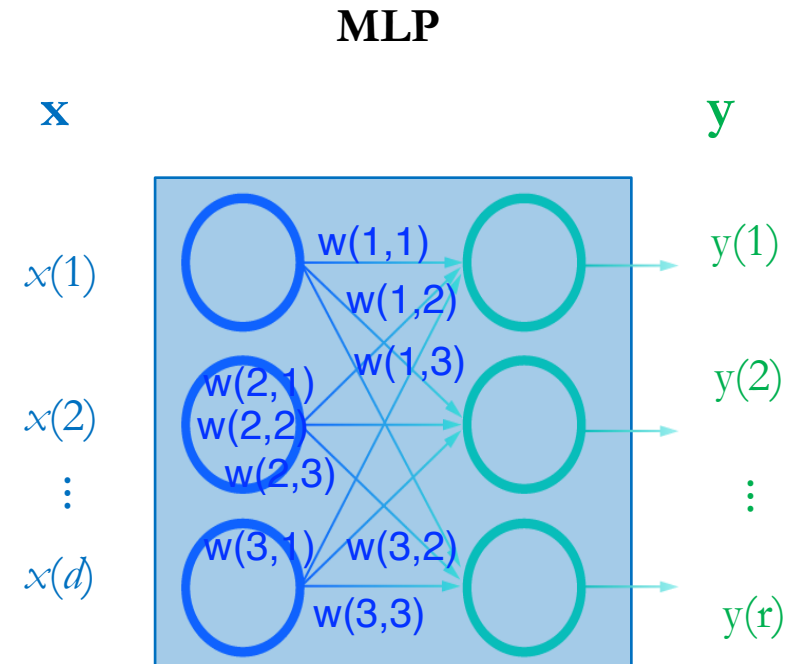
$$\mathbf{y} = g(\mathbf{W}\mathbf{x} + \mathbf{b})$$

où :

$$\mathbf{W} = \begin{bmatrix} w_{x1,1} & \dots & w_{xd,1} \\ w_{x1,2} & \dots & w_{xd,2} \\ \vdots & \ddots & \vdots \\ w_{x1,r} & \dots & w_{xd,r} \end{bmatrix}, \mathbf{x} = \begin{bmatrix} x(1) \\ x(2) \\ \vdots \\ x(d) \end{bmatrix}, \mathbf{y} = \begin{bmatrix} y(1) \\ y(2) \\ \vdots \\ y(r) \end{bmatrix}, \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_r \end{bmatrix}$$

où g est la fonction d'activation, d est la dimension d'entrée, r est le nombre de neurones dans la couche cachée (la dimension du vecteur de sortie y).

La même chose peut être écrite pour toutes les couches cachées.



ex: $y(2) = f(x(1)*w(1,2) + x(2)*w(2,2) + x(3)*w(3,2))$

Réseau neuronal récurrent (RNN)

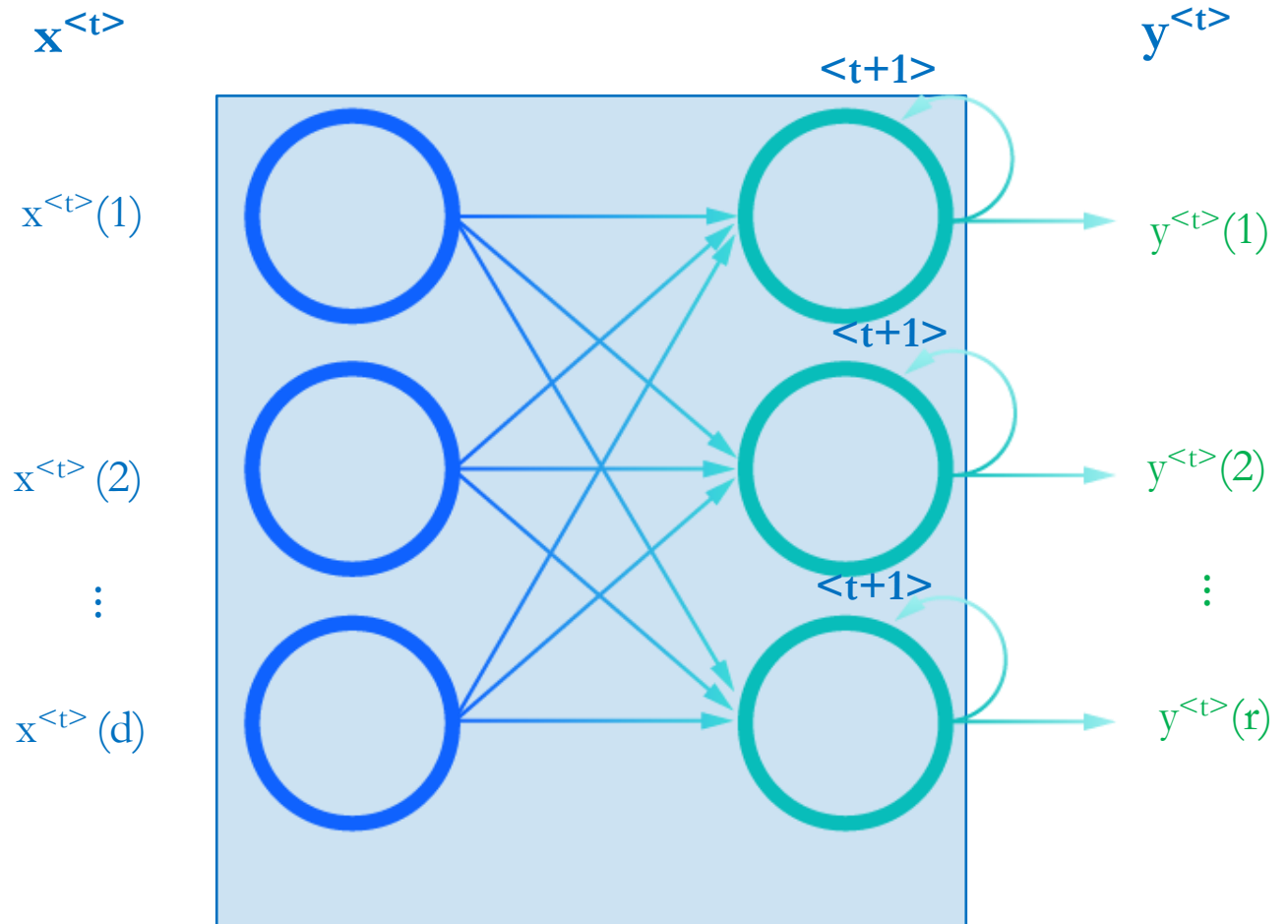
La structure d'une couche RNN est similaire à celle d'un MLP en ce sens qu'elle reçoit une entrée de dimension d au pas de temps t (ici, $\mathbf{x}^{<t>}$) et produit une sortie de dimension r au pas de temps t (ici, $\mathbf{y}^{<t>}$).

La principale différence est que l'entrée est la séquence temporelle et qu'il existe T entrées de dimension d , c'est-à-dire $\mathbf{x}^{<1>}, \mathbf{x}^{<2>}, \dots, \mathbf{x}^{<T>}$.

De plus, il existe des connexions de rétroaction de chaque cellule à elle-même qui forment la mémoire du réseau. **Nous avons donc deux types de connexions : une connexion directe, comme MLP, et une connexion rétroactive.**

Remarque : la rétroaction n'est pas nécessairement la sortie ; il s'agit d'un état caché (hidden state).

RNN à la séquence temporelle $<t>$



En supposant T séquences temporelles, l'architecture RNN ci-dessus peut être décomposée en T modules

Réseau neuronal récurrent (RNN)

La figure supérieure à droite illustre une couche de Réseau Neuronal Récurrent (RNN) dépliée.

Chaque carré bleu représente une cellule RNN qui reçoit le vecteur d'entrée $\mathbf{x}^{<t>}$ et le vecteur d'état caché $\mathbf{a}^{<t-1>}$ et génère le vecteur d'état caché suivant $\mathbf{a}^{<t>}$ et le vecteur de sortie $\mathbf{y}^{<t>}$.

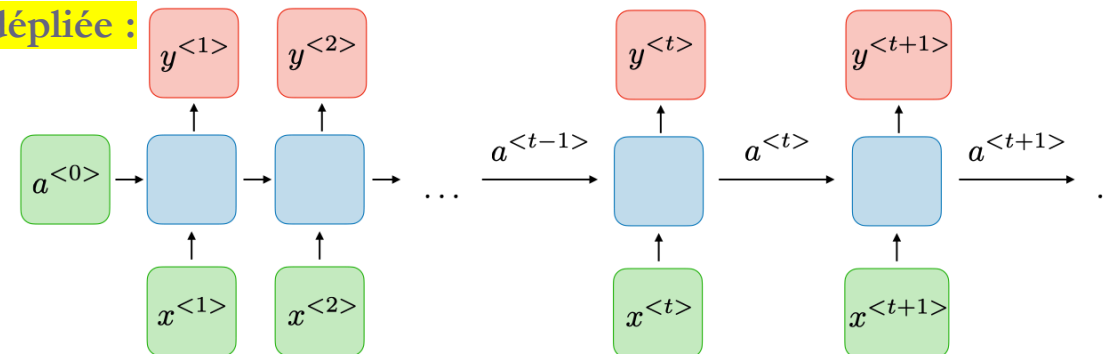
Attention : Les états cachés $\mathbf{a}^{<t>}$ constituent la mémoire du réseau et peuvent être différents des sorties $\mathbf{y}^{<t>}$.

Les états cachés sont calculés, à l'aide d'une transformation de type feed-forward, comme une combinaison linéaire des entrées et des états cachés précédents (matrices de poids $\mathbf{W}_{aa}$, $\mathbf{W}_{ax}$ et vecteur de biais $\mathbf{b}_a$), suivie d'une fonction d'activation non linéaire g_1 .

Les sorties sont les estimations finales du réseau (de la cible du réseau) et sont calculées, à l'aide d'une transformation de type feed-forward, à partir des états cachés (matrice de poids $\mathbf{W}_{ya}$ le vecteur de biais $\mathbf{b}_y$) suivie d'une fonction d'activation non linéaire g_2 .

En résumé, une couche de RNN nécessite trois matrices de poids pour sa transformation, contre une seule matrice de poids pour une couche du MLP.

Une couche de RNN, dépliée :



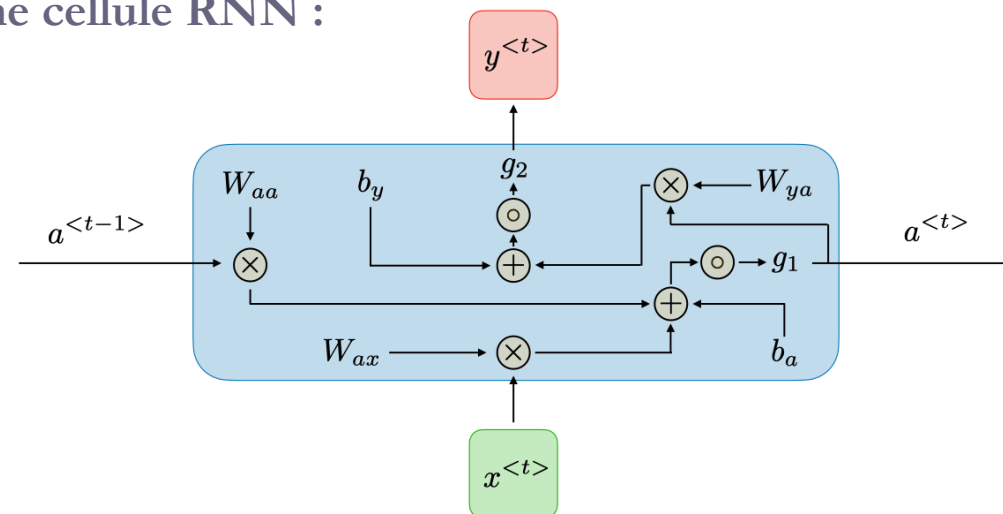
Hidden state:

$$\mathbf{a}^{<t>} = g_1(\mathbf{W}_{aa}\mathbf{a}^{<t-1>} + \mathbf{W}_{ax}\mathbf{x}^{<t>} + \mathbf{b}_a)$$

Output:

$$\mathbf{y}^{<t>} = g_2(\mathbf{W}_{ya}\mathbf{a}^{<t>} + \mathbf{b}_y)$$

Une cellule RNN :

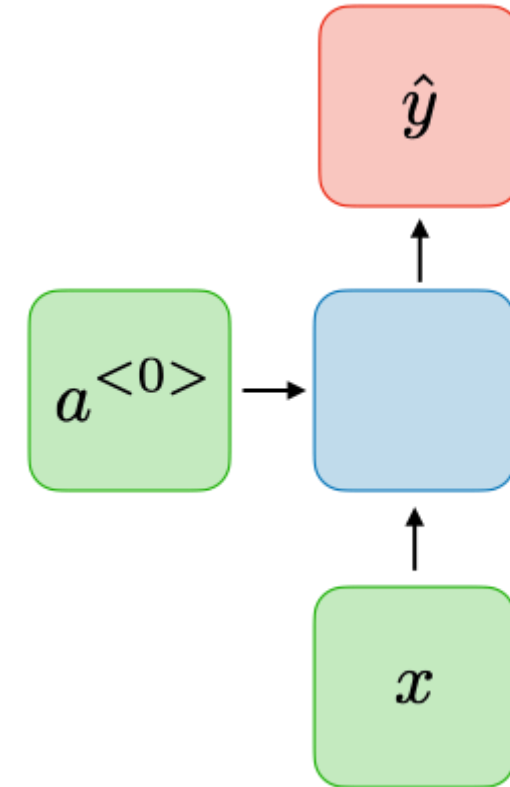


Types de RNN

One-to-one (Un-à-un)

Il s'agit du réseau de neurones typique où il n'existe qu'une seule séquence d'entrée.

C'est essentiellement le réseau de neurones traditionnel (MLP), car il n'y a pas de séquence temporelle.



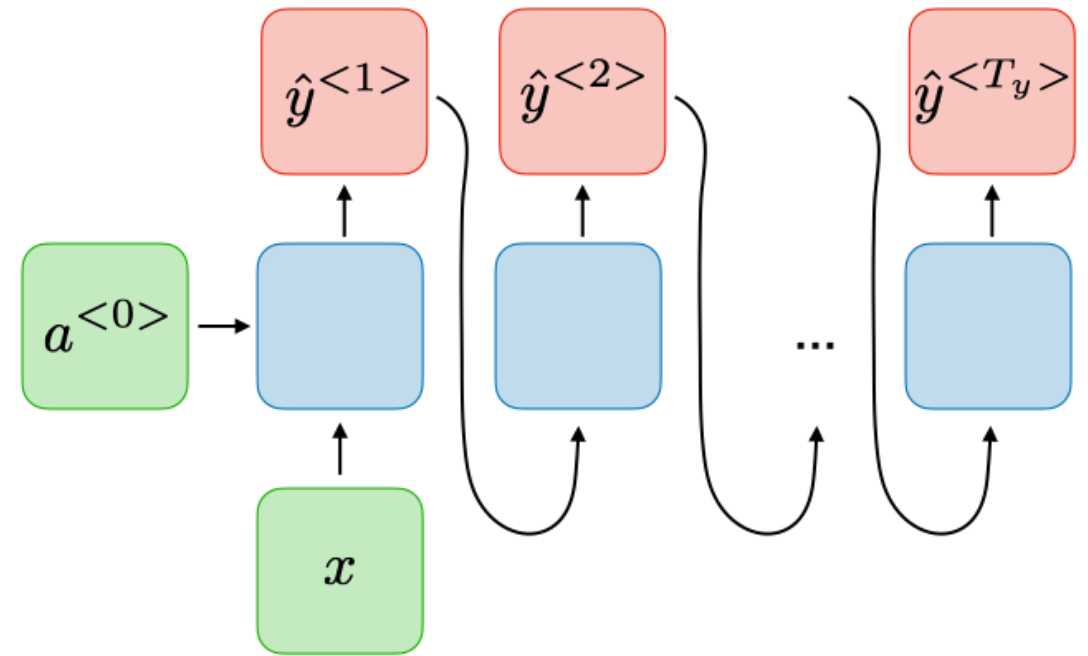
Types de RNN

One-to-many (Un-à-plusieurs)

Il n'y a qu'un seul pas de temps d'entrée, mais plusieurs pas de temps de sortie.

Cette architecture est principalement utilisée lorsque l'on souhaite synthétiser ou générer une séquence de sorties.

Dans les applications biomédicales, elle peut être utilisée pour prévoir plus loin dans le futur (l'apparition d'un événement ou la valeur d'une quantité) ou pour prédire ou générer une forme d'onde.

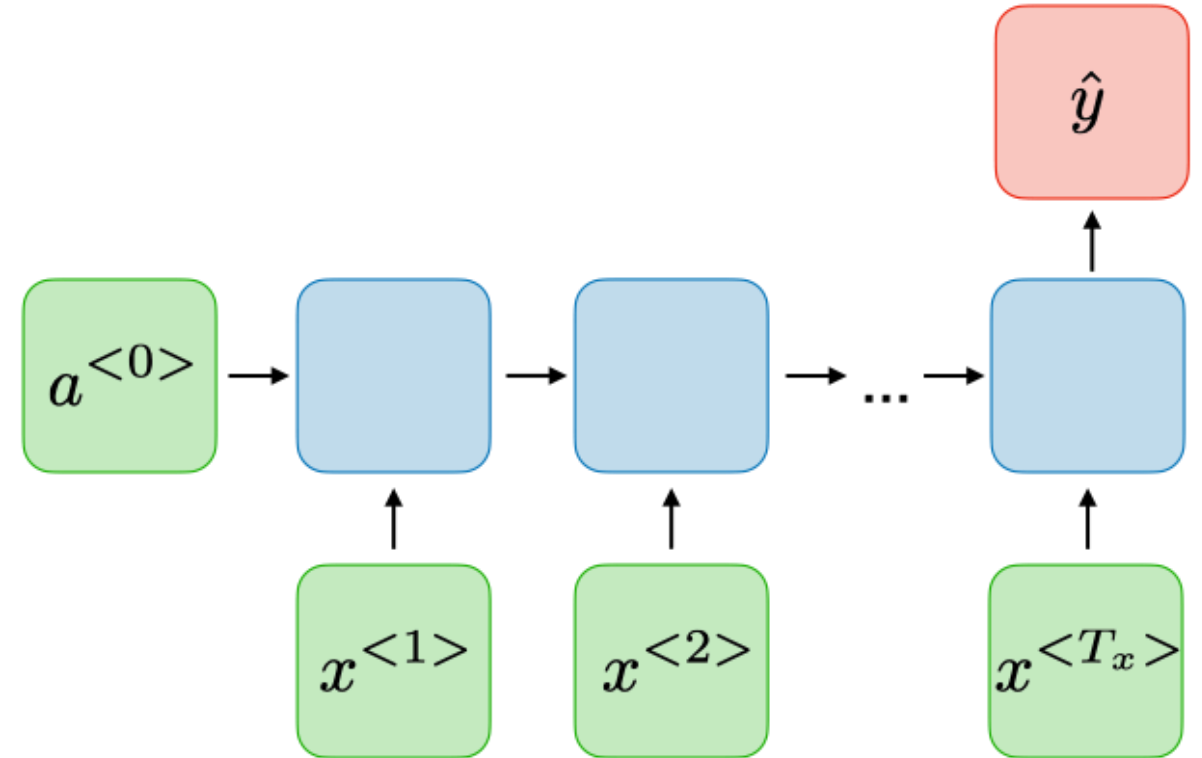


Utilisé pour la generation des donnes

Types de RNN

Many-to-one (Plusieurs à un)

Peut être utilisé dans la plupart des tâches de prédiction médicale.

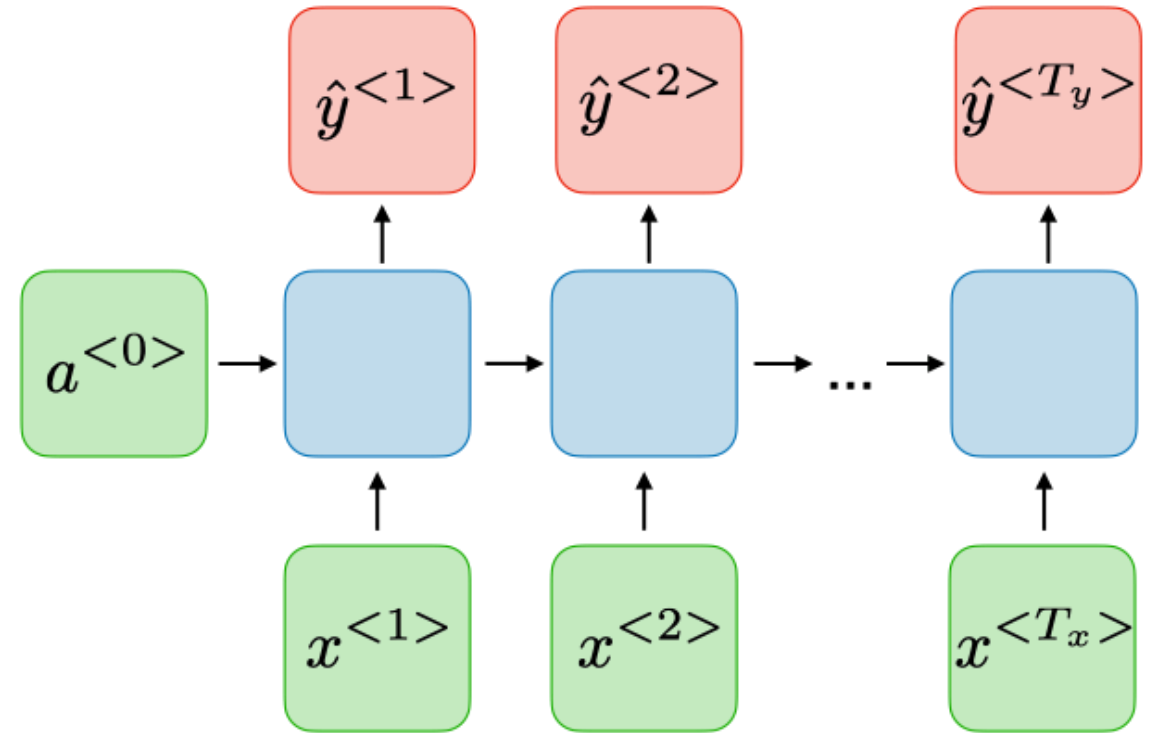


Ici on utilise une sequence temporelle. Chaque sortie de chaque partie est utilisé comem le $a^{<i>}$ pour le prcogain etape

Types de RNN

Many-to-many (Plusieurs à plusieurs)

Peut être utilisé dans les tâches d'estimation et de détection.



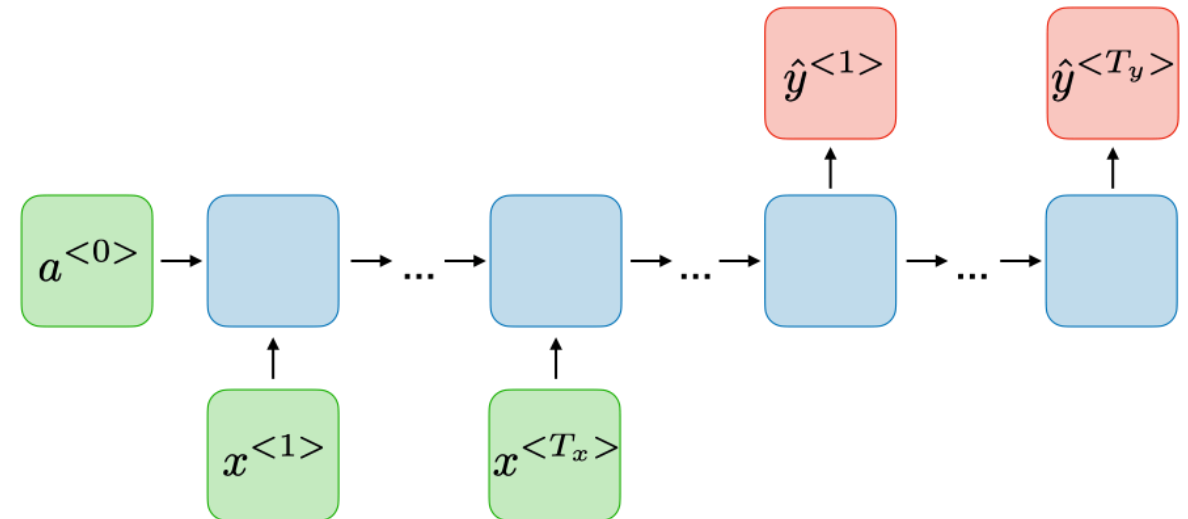
Ici on utilise une sequence temporelle et on a une sortie estimé pour chaque element de la sequence temporelle

Types de RNN

Many-to-many (Plusieurs à plusieurs)

Le modèle lit d'abord une séquence d'entrée complète (passé ou contexte), puis génère une séquence de sorties plus tard dans le temps

Peut être utilisé dans des tâches de prévision à différents intervalles dans le futur.



Fonction de coût

La fonction de coût pour les réseaux neuronaux récurrents à tous les pas de temps est définie sur la base de la perte à chaque pas de temps :

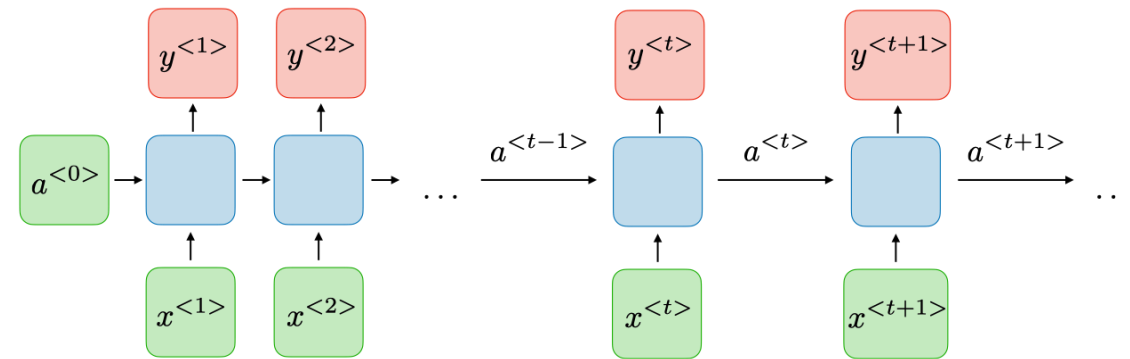
$$L^{Overall} = \sum_{t=1}^{T_y} L^{<t>}$$

où

$$L^{<t>} = L(y^{<t>}, \hat{y}^{<t>})$$

T_y est le nombre de séquences temporelles de sortie et $L^{<t>}$ est la perte au pas de temps t .

La perte globale est donc la somme des pertes de chacun des pas de temps.

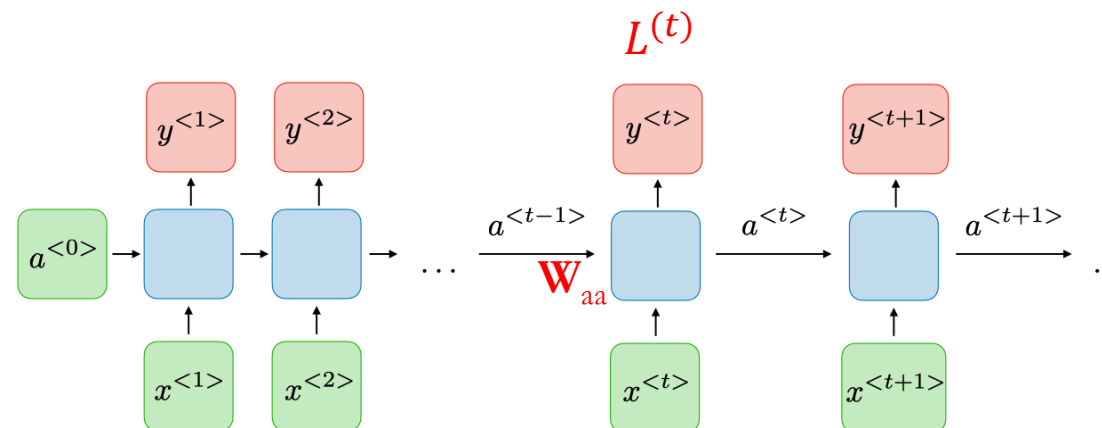


$$L^{<t>} = L(y^{<t>}, \hat{y}^{<t>}) = y^{<t>} - \hat{y}^{<t>}$$

Rétropropagation dans le temps (BPTT)

En se basant sur la fonction de coût déjà établie, le gradient pour un réseau neuronal récurrent est calculé à chaque pas de temps, puis sommé sur tous les pas de temps, et enfin minimisé par un processus connu sous le nom de rétropropagation à travers le temps (BPTT), qui implique le calcul des gradients des états cachés par rapport aux états cachés précédents ; notamment, les nombreuses multiplications inhérentes à ce processus, comme illustré dans les calculs de gradient à l'étape de temps t , peuvent entraîner **le problème de disparition du gradient**.

Les calculs sont destinés aux curieux.



Rappel :

$$\mathbf{a}^{<t>} = g_1(\mathbf{W}_{aa}\mathbf{a}^{<t-1>} + \mathbf{W}_{ax}\mathbf{x}^{<t>} + \mathbf{b}_a)$$

$$\mathbf{a}^{<t-1>} = g_1(\mathbf{W}_{aa}\mathbf{a}^{<t-2>} + \mathbf{W}_{ax}\mathbf{x}^{<t-1>} + \mathbf{b}_a)$$

En utilisant la règle de la chaîne, nous pouvons montrer que :

$$\frac{\partial L^{<t>}}{\partial \mathbf{W}_{aa}} = \frac{\partial L^{<t>}}{\partial y^{<t>}} \cdot \frac{\partial y^{<t>}}{\partial \mathbf{a}^{<t>}} \cdot \frac{\partial \mathbf{a}^{<t>}}{\partial \mathbf{W}_{aa}}$$

où

$$\frac{\partial \mathbf{a}^{<t>}}{\partial \mathbf{W}_{aa}} = \frac{\partial g_1^{<t>}}{\partial \mathbf{W}_{aa}} + \frac{\partial g_1^{<t>}}{\partial \mathbf{a}^{<t-1>}} \frac{\partial \mathbf{a}^{<t-1>}}{\partial \mathbf{W}_{aa}}$$

On peut démontrer que (? démontrer pour $t = 3$) :

$$\frac{\partial \mathbf{a}^{<t>}}{\partial \mathbf{W}_{aa}} = \frac{\partial g_1^{<t>}}{\partial \mathbf{W}_{aa}} + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^t \frac{\partial g_1^{<j>}}{\partial \mathbf{a}^{<j-1>}} \right) \cdot \frac{\partial g_1^{<i>}}{\partial \mathbf{W}_{aa}}$$

Si vous êtes intéressé, cliquez ici pour plus de détails :

https://d2l.ai/chapter_recurrent-neural-networks/bptt.html

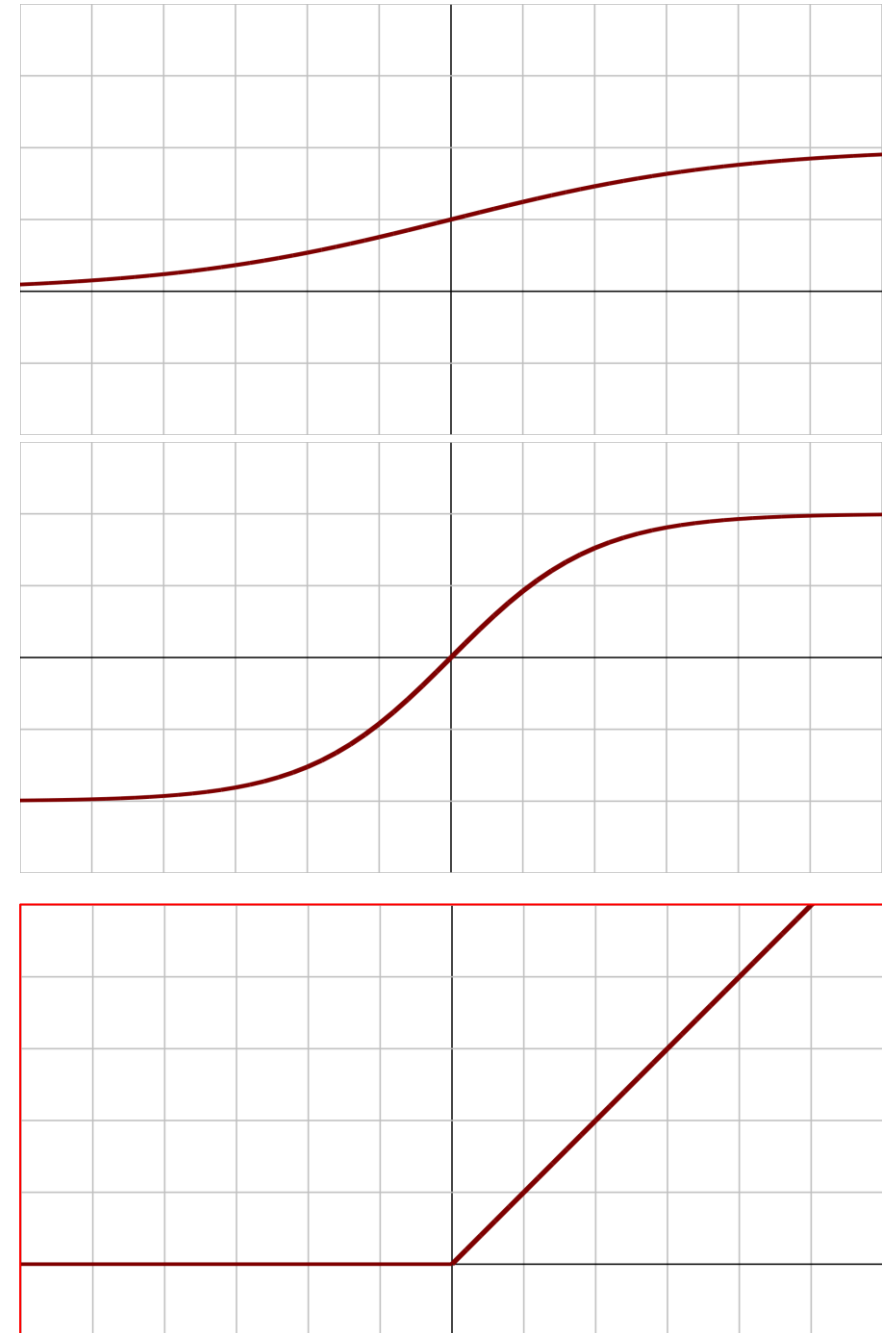
Choix de la fonction d'activation

Éviter la disparition du gradient

Similaire aux MLP, nous avons plusieurs choix pour les fonctions d'activation.

Un avantage de **ReLU** est que sa dérivée est 1, ce qui peut aider à éviter le problème de disparition du gradient. SOS

Elle permet également à certains neurones de rester inactifs, ce qui peut améliorer l'apprentissage global, en particulier dans les réseaux plus profonds (agissant comme une forme de régularisation pour éviter le surajustement).



Mémoire à long terme (LSTM)

Évitez la disparition du gradient en modifiant l'architecture du RNN

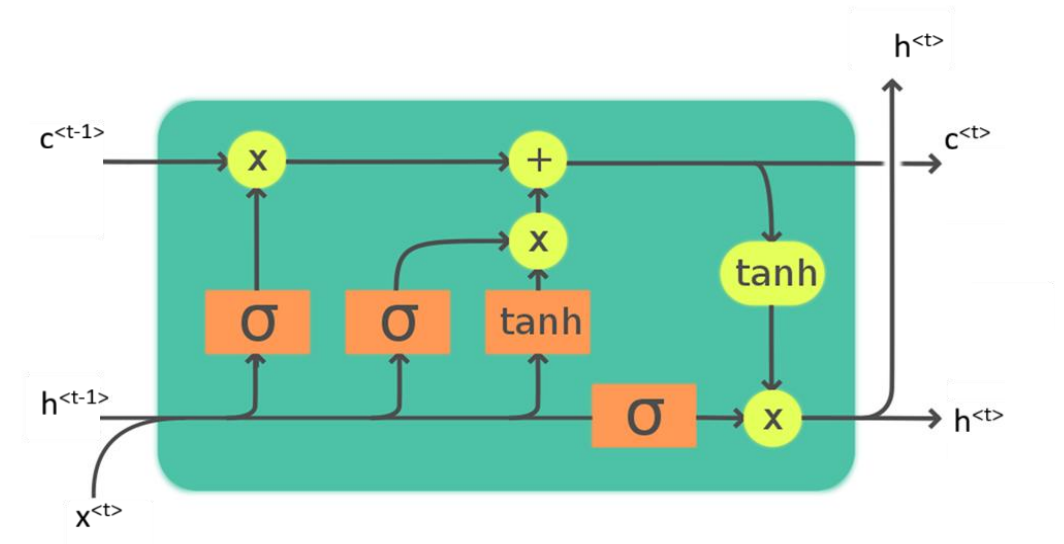
La LSTM est un RNN utilisant plusieurs portes pour contrôler le flux d'informations lors du traitement de séquences.

Les variables clés de la LSTM incluent :

Entrée $x^{<t>}$: Données d'entrée de l'étape de temps actuelle.

État caché (Hidden state) $h^{<t>}$: Mémoire à court terme (état caché), produisant à chaque étape et transmettant des infos (Il s'agit de la sortie).

État cellulaire (Cell state) $c^{<t>}$: Mémoire à long terme (état de la cellule), conservant des infos sur les séquences et influençant les états futurs.



Legend:

Layer	Componentwise operation	Copy	Concatenate

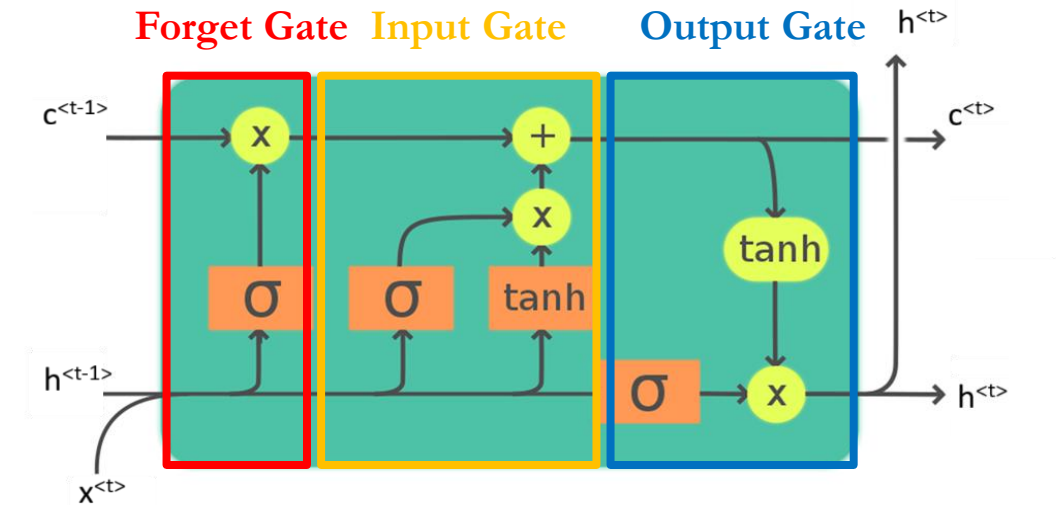
Mémoire à long terme (LSTM)

Un LSTM typique comporte trois portes :

- Porte d'oubli (Forget Gate) : décide ce qui est pertinent de conserver de l'état de cellule précédent $c^{<t-1>}$
- Porte d'entrée (Input Gate) : décide quelles nouvelles informations provenant de l'entrée actuelle $x^{<t>}$ doivent être ajoutées à l'état de cellule $c^{<t>}$
- Porte de sortie (Output Gate) : Décide quelle partie de l'état de cellule mis à jour $c^{<t>}$ sera utilisée pour calculer la sortie de la cellule (l'état caché $h^{<t>}$)

Ces portes ont chacune leur propre réseau neuronal et agissent comme des filtres qui traitent l'information.

Attention: Dans un LSTM, la fonction **sigmoïde** (σ) est utilisée pour créer des filtres qui régulent sélectivement le flux d'information (décidant quoi conserver ou oublier), tandis que la fonction **tanh** est utilisée pour générer le contenu candidat qui sera ajouté à la mémoire interne.



Legend:

Layer	Componentwise operation	Copy	Concatenate

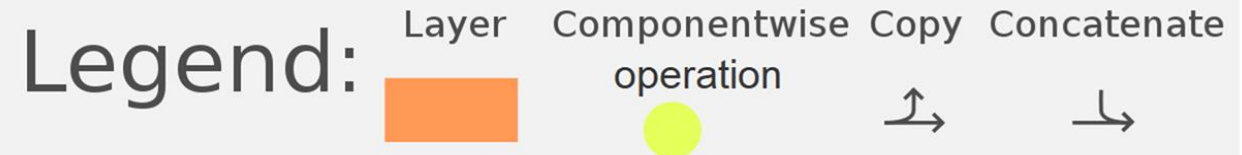
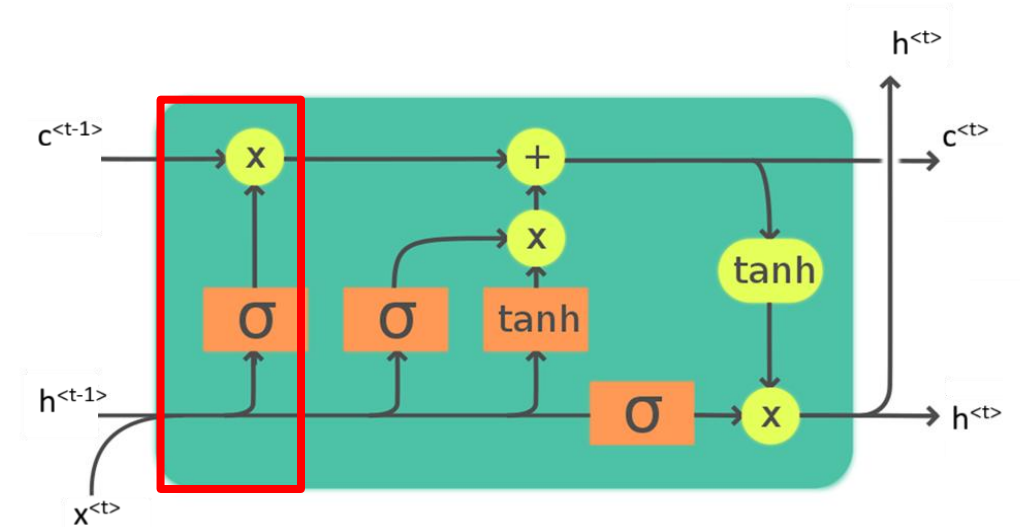
LSTM

Porte d'oubli (Forget Gate):

La porte d'oubli décide ce qui est pertinent de conserver de l'état de cellule précédent ($\mathbf{c}^{<t-1>}$) (mémoire à long terme) étant donné à la fois l'état caché précédent ($\mathbf{h}^{<t-1>}$) et les nouvelles données d'entrée $\mathbf{x}^{<t>}$.

L'état caché précédent ($\mathbf{h}^{<t-1>}$) et les données d'entrée actuelles ($\mathbf{x}^{<t>}$) sont concaténés, puis combinés par une transformation linéaire ($\mathbf{W}_f$) suivie d'une activation sigmoïde (comme une couche à propagation avant), qui produit un vecteur de valeurs comprises entre 0 et 1 (grâce à la sigmoïde). Chaque valeur agit comme un filtre : plus elle est proche de 1, plus elle laisse passer l'information.

Ces valeurs de sortie sont ensuite envoyées et multipliées point par point (Produit de Hadamard $\odot$) avec l'état de cellule précédent $\mathbf{c}^{<t-1>}$. Cette multiplication point par point signifie que les composantes de l'état cellulaire qui ont été jugées non pertinentes par le réseau de la porte d'oubli seront multipliées par un nombre proche de 0 et auront donc moins d'influence sur les étapes suivantes.



$$\mathbf{f}^{<t>} = \sigma(\mathbf{W}_f [\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_f)$$

$$\mathbf{c}'^{<t-1>} = \mathbf{f}^{<t>} \odot \mathbf{c}^{<t-1>}$$

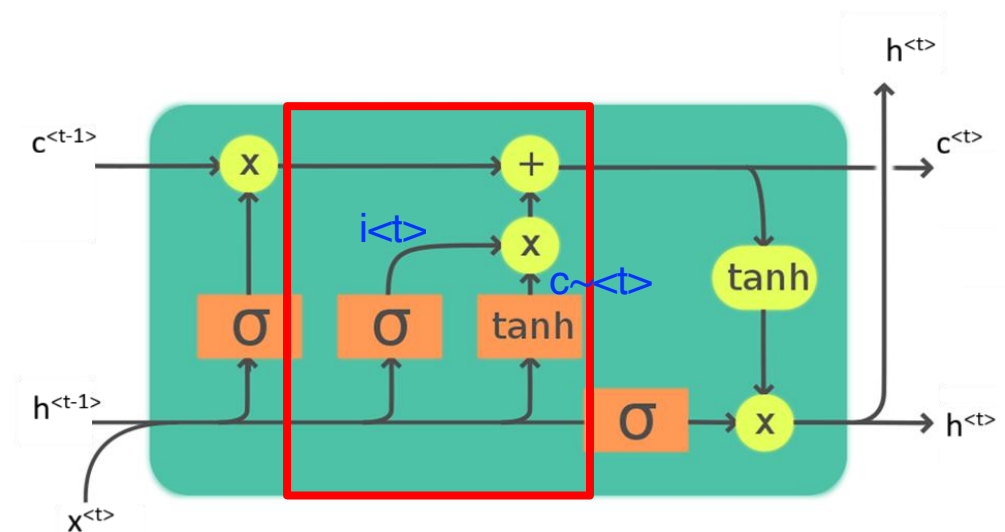
LSTM

Nouveau réseau de mémoire (New memory network) :

La **nouvelle mémoire candidate** $\mathbf{c}^{<t>}$ est calculée à l'aide d'une transformation linéaire appliquée à l'état caché précédent $\mathbf{h}^{<t-1>}$ et à l'entrée courante $\mathbf{x}^{<t>}$, suivie d'une activation **tanh** (comme une couche à propagation avant). On obtient ainsi un vecteur candidat de nouvelles informations $\tilde{\mathbf{c}}^{<t>}$ susceptibles d'être ajoutées à l'état de cellule (mémoire à long terme).

Cependant, ce n'est pas directement ce vecteur qui modifie la mémoire. C'est la **porte d'entrée** $\mathbf{i}^{<t>}$ — calculée elle aussi par une transformation linéaire + activation sigmoïde — qui décide dans quelle mesure la mémoire candidate $\tilde{\mathbf{c}}^{<t>}$ est effectivement intégrée dans l'état de cellule.

La fonction tanh est utilisée car elle contraint les valeurs entre -1 et 1, ce qui permet au modèle non seulement d'ajouter de nouvelles informations, mais aussi de retirer ou corriger certaines composantes de la mémoire si nécessaire.



Legend:

Layer	Componentwise operation	Copy	Concatenate

$$\mathbf{i}^{<t>} = \sigma(\mathbf{W}_i [\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_i)$$

$$\tilde{\mathbf{c}}^{<t>} = \tanh(\mathbf{W}_c [\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_c)$$

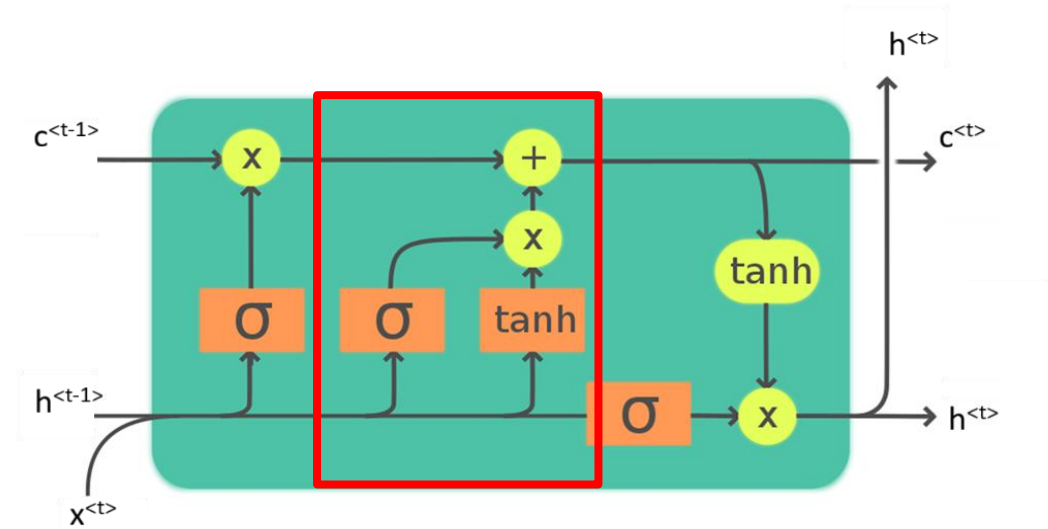
$$\mathbf{c}^{<t>} = \mathbf{f}^{<t>} \odot \mathbf{C}^{<t-1>} + \mathbf{i}^{<t>} \odot \tilde{\mathbf{c}}^{<t>}$$

LSTM

La porte d'entrée (Input gate) :

La sortie du nouveau réseau de mémoire et celle de la porte d'entrée sont multipliées élément par élément. Cela permet de filtrer et d'éliminer les informations inutiles du vecteur de mémoire candidat.

Le vecteur ainsi filtré est ensuite ajouté à la sortie de la porte d'oubli, ce qui met à jour la mémoire à long terme $\mathbf{c}^{<t>}$ du réseau.



Legend:

Layer	Componentwise operation	Copy	Concatenate

$$\mathbf{i}^{<t>} = \sigma(\mathbf{W}_i [\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_i)$$

$$\tilde{\mathbf{c}}^{<t>} = \tanh(\mathbf{W}_c [\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_c)$$

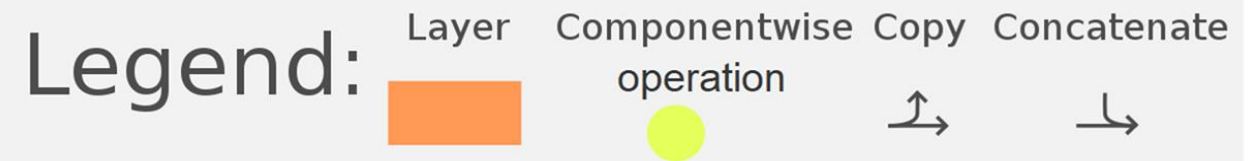
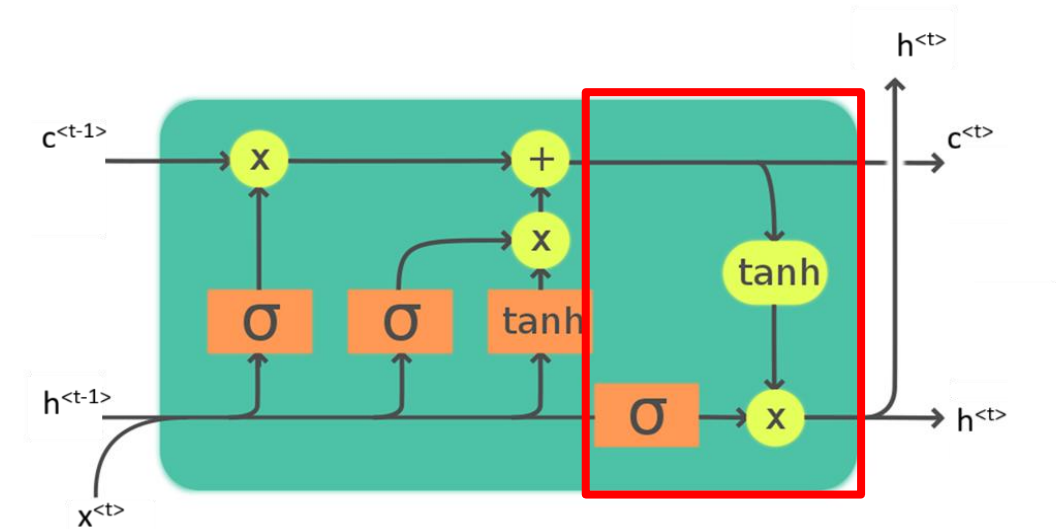
$$\mathbf{c}^{<t>} = \mathbf{f}^{<t>} \odot \mathbf{c}^{<t-1>} + \mathbf{i}^{<t>} \odot \tilde{\mathbf{c}}^{<t>}$$

LSTM

La porte de sortie (output gate) :

Après la mise à jour de la mémoire à long terme $\mathbf{c}^{<t>}$, la dernière étape est de calculer l'état caché $\mathbf{h}^{<t>}$ grâce à la porte de sortie.

Pour ce faire, nous créons un filtre $\mathbf{o}_t$ en utilisant $\mathbf{h}^{<t-1>}$ et $\mathbf{x}^{<t>}$. Toutefois, avant d'appliquer le filtre à la mémoire mise à jour $\mathbf{c}^{<t>}$, on passe $\mathbf{c}^{<t>}$ par une fonction $\tanh$ pour limiter ses valeurs entre $[-1,1]$. Cela permet d'avoir un état caché $\mathbf{h}^{<t>}$ contrôlé et borné.



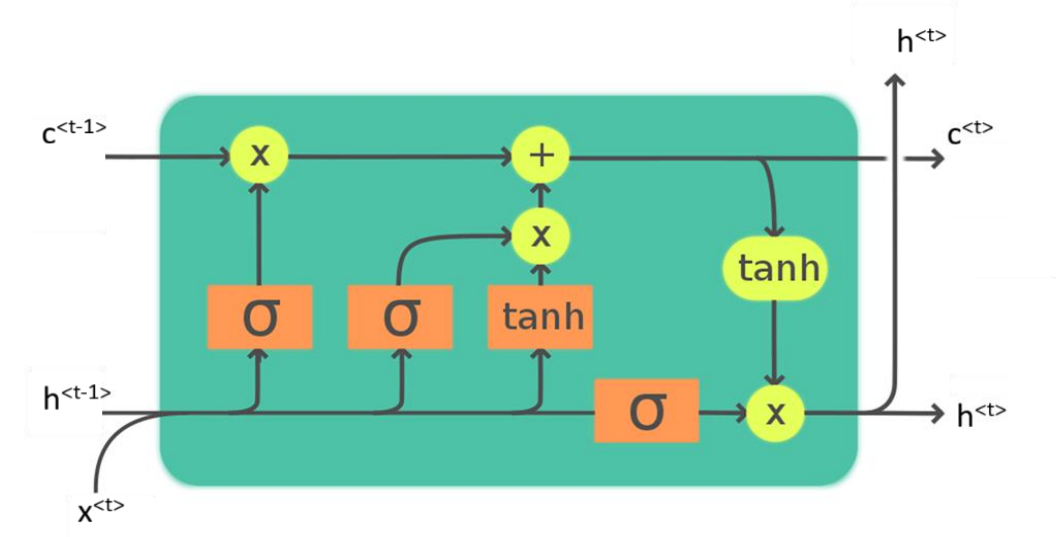
$$\mathbf{o}^{<t>} = \sigma(\mathbf{W}_o [\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_o)$$

$$\mathbf{h}^{<t>} = \mathbf{o}^{<t>} \odot \tanh(\mathbf{c}^{<t>})$$

LSTM

Remarque :

La sortie du LSTM est l'état caché $h^{<t>}$. Mais, dans la pratique, $h^{<t>}$ est transmis à une couche MLP entièrement connectée afin de lui donner une puissance supplémentaire pour la prédiction de n'importe quel type de sortie.



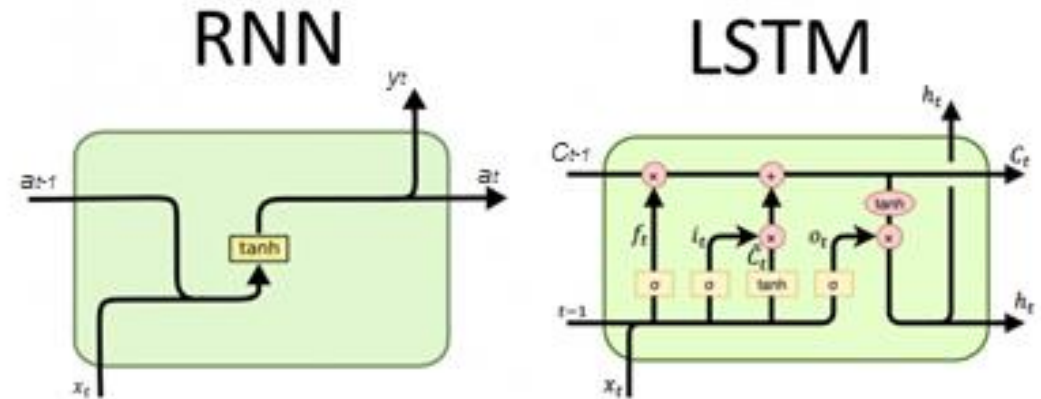
Legend:

Layer	Componentwise operation	Copy	Concatenate

RNN vs LSTM

Grâce à son mécanisme de portes :

- Les LSTM peuvent mieux capturer les dépendances à long terme comparé aux RNN simples.
- Les LSTM peuvent mettre en avant de manière sélective les parties les plus importantes de la séquence d'entrée.
- Ce mécanisme de portes aide à atténuer les problèmes de gradients qui disparaissent en introduisant des interactions additives en plus des interactions multiplicatives.
- Cependant, les LSTM ont plus de paramètres que les RNN simples, ce qui les rend plus susceptibles au surapprentissage.



Unité Récurrente Portée

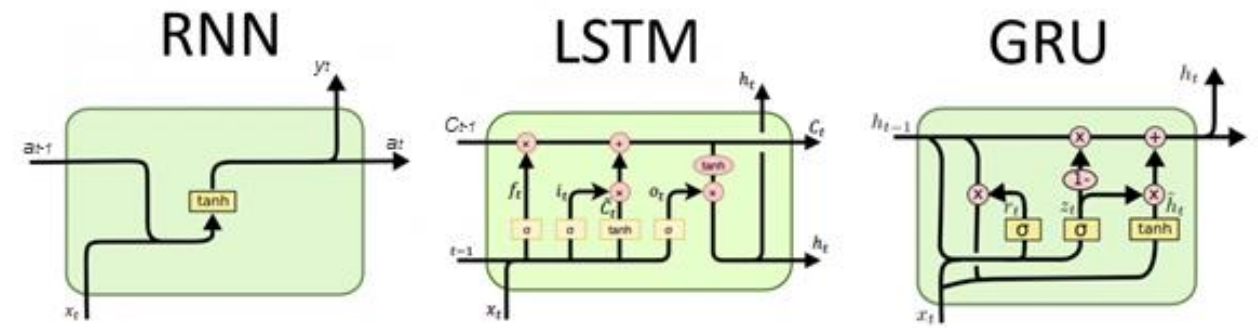
Gated recurrent unit (GRU)

La principale différence entre le GRU et le LSTM est que le GRU possède deux portes qui sont la réinitialisation et la mise à jour, tandis que le LSTM possède trois portes qui sont l'entrée, la sortie et l'oubli.

Les portes de réinitialisation et de mise à jour décident de l'information qui sera transmise aux cellules suivantes. Vous trouverez ici un didacticiel simple sur le fonctionnement de la GRU :

<https://towardsdatascience.com/understanding-gru-networks-2ef37df6c9be>

La GRU est moins complexe que la LSTM, car elle comporte un plus petit nombre de portes. Si l'ensemble de données est petit, la GRU est préférable à la LSTM pour les ensembles de données plus importants.

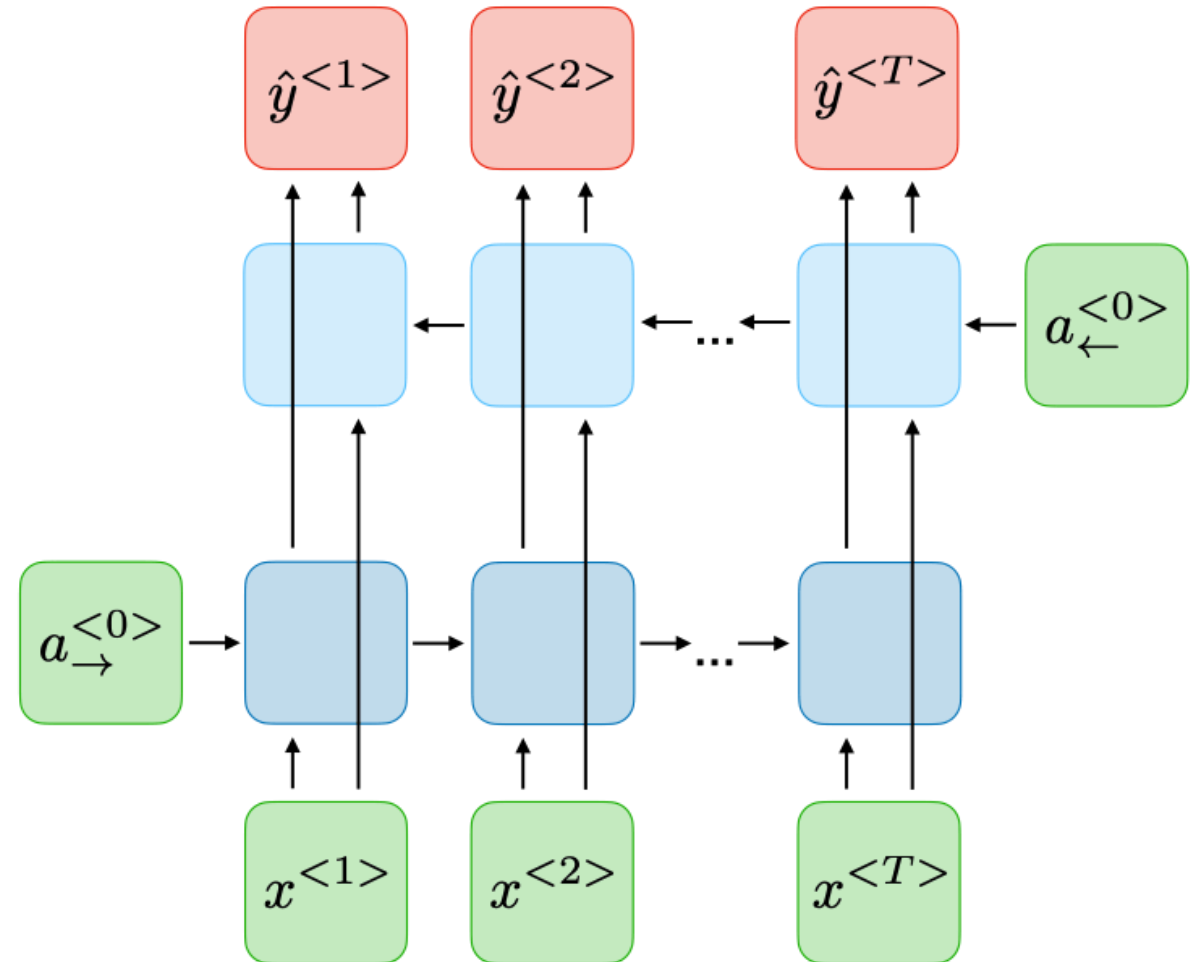


Variantes de RNN

RNN bidirectionnel

Ce réseau facilite encore l'apprentissage des modèles temporels en examinant le flux d'informations à la fois dans le sens avant et dans le sens arrière.

Nous avons encore plus des paramètres que le LSTM de base, donc ici on risque encore plus d'avoir un surapprentissage

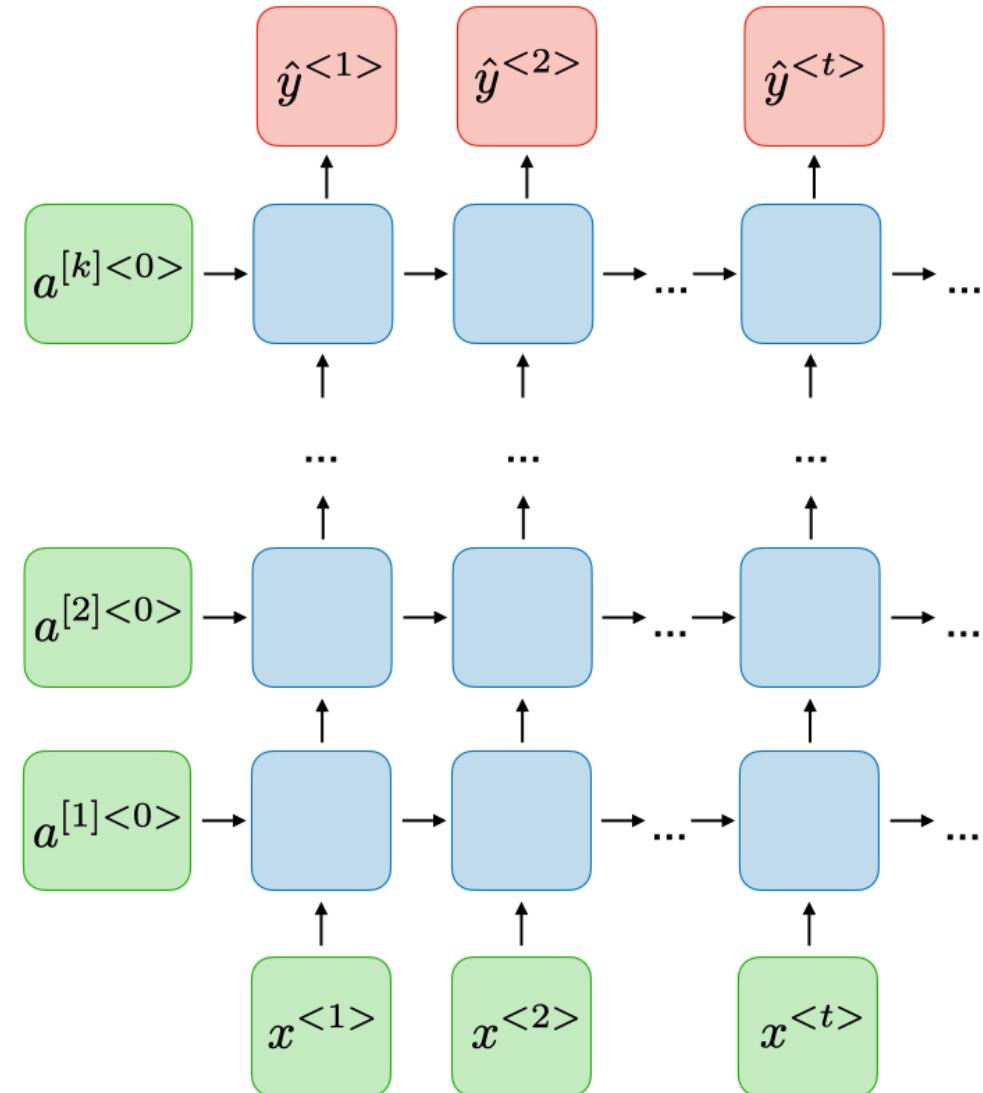


Variantes de RNN

RNN profond

Les RNN profonds peuvent être construits en ajoutant simplement des couches supplémentaires au réseau. Cela peut aider le réseau à extraire des caractéristiques utiles et à apprendre des modèles temporels plus complexes.

Cependant, plus la complexité augmente, plus il faut de données pour entraîner le réseau et éviter le surajustement.



Exercice 2

Concevez une LSTM qui peut être utilisée pour notre problème de prédiction de BGL.

Comment formeriez-vous les entrées et quelle forme de RNN utiliseriez-vous ?

Quelles sont les dimensions des entrées et des sorties ?

Exercice 2

Concevez une LSTM qui peut être utilisée pour notre problème de prédiction de BGL.

Comment formeriez-vous les entrées et quelle forme de RNN utiliseriez-vous ?

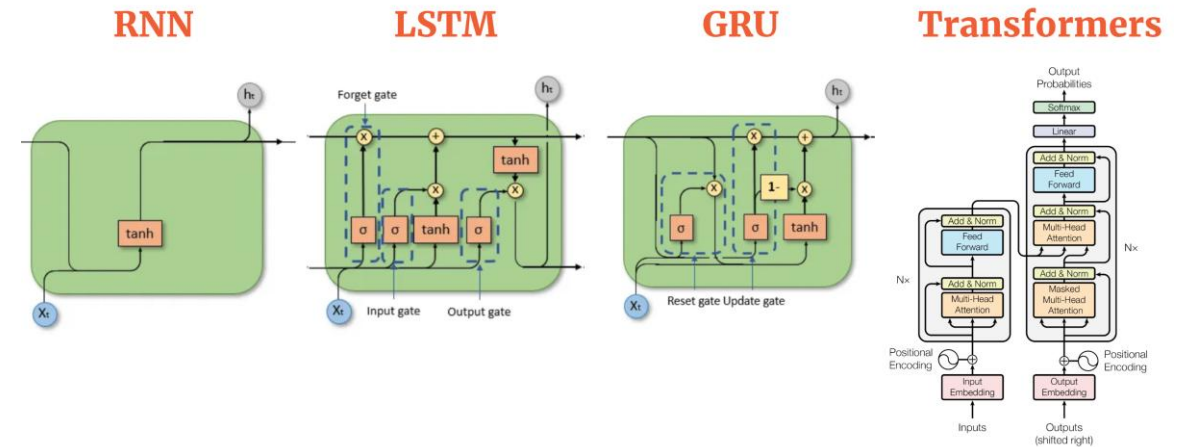
Quelles sont les dimensions des entrées et des sorties ?

Transformers

Bien que les LSTM aient révolutionné la modélisation des séquences grâce à leur capacité à saisir les dépendances à long terme, elles peinent face aux relations temporelles excessivement longues et leur nature séquentielle intrinsèque limite leur vitesse de traitement.

Limitation	Pourquoi
Dépendances temporelles excessivement longues	L'information est « diluée » ou submergée dans la cellule de mémoire, et la propagation du gradient demeure instable sur des milliers d'étapes séquentielles.
Vitesse de traitement limitée	La nature séquentielle inhérente oblige à un calcul étape par étape, ce qui empêche le traitement parallèle (contrairement aux modèles comme les Transformers).

Les **Transformers** résolvent ces problèmes grâce à un **mécanisme d'auto-attention** permettant un traitement parallèle et une meilleure compréhension du contexte global.



<https://aiml.com/compare-the-different-sequence-models-rnn-lstm-gru-and-transformers/>

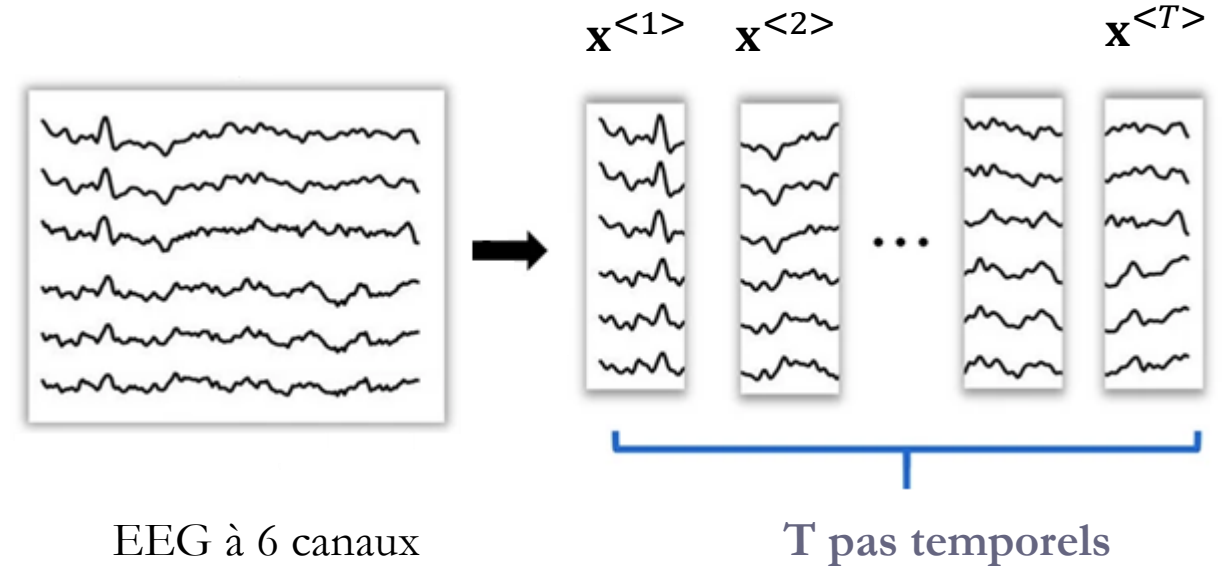
Transformers

Considérons une séquence de **T pas temporels** (ou **T tokens**) $\{\mathbf{x}^{<t>}\}_{t=1}^T$, chacun représenté par un vecteur de caractéristiques (*embedding*) de dimension d . Cela constitue un exemple de données d'entrée pour entraîner notre Transformer. (Afin d'entraîner efficacement le modèle, nous avons besoin de nombreux exemples similaires.)

Nous recherchons une représentation enrichie $\{\mathbf{y}^{<t>}\}_{t=1}^T$, qui tient compte des dépendances à long terme et des interactions complexes dans le signal biomédical.

Par exemple,

- Les transitions entre les stades du sommeil dépendent de schémas et d'événements (comme les fuseaux ou les complexes K dans l'EEG) qui s'étalent sur plusieurs minutes.
- Ou encore, dans la prédiction des crises d'épilepsie : Les modifications de l'EEG peuvent précéder une crise jusqu'à une heure, ce qui nécessite la détection de dynamiques subtiles, émergeant lentement.



L'idée derrière Transformer est d'extraire une présentation plus riche des données temporelles. Celle-ci peut ensuite être utilisée par un autre réseau pour des tâches de prédiction.

Transformers

T pas temporels (ou T *tokens*): $\{\mathbf{x}^{<t>}\}_{t=1}^T$,

Nous recherchons une représentation enrichie : $\{\mathbf{y}^{<t>}\}_{t=1}^T$

Pour ce faire, le transformer applique un mécanisme d'attention, qui permet à chaque pas t de la séquence de pondérer (faire attention) dynamiquement les informations provenant des autres pas temporels.

Par exemple, pour l'étape t , je peux écrire que $\mathbf{y}^{<t>}$ (notre nouvelle représentation pour $\mathbf{x}^{<t>}$) est une somme pondérée des autres étapes :

$$\mathbf{y}^{<t>} = a_{t1}\mathbf{x}^{<1>} + \dots + a_{tm}\mathbf{x}^{<m>} + \dots + a_{tT}\mathbf{x}^{<T>} = \sum_{m=1}^T a_{tm}\mathbf{x}^{<m>} \quad \text{le vecteur value} \quad \uparrow$$

où a_{tm} représente le poids (**quantité d'attention**) que $\mathbf{x}_t$ accorde à $\mathbf{x}_m$.

Avec les contraintes : $a_{tm} \geq 0$ et $\sum_{m=1}^T a_{tm} = 1$

Nous représentons la séquence temporels dans un format matriciel :

$$X_{T \times d} = [\mathbf{x}^{<1>}, \dots, \mathbf{x}^{<t>}, \dots, \mathbf{x}^{<T>}]^{\text{tr}}$$

Le Transformer produit une nouvelle représentation dans un espace transformé :

$$Y_{T \times d} = [\mathbf{y}^{<1>}, \dots, \mathbf{y}^{<t>}, \dots, \mathbf{y}^{<T>}]^{\text{tr}}$$

Pour calculer le coefficient a_{tm} , on compare $\mathbf{x}^{<t>}$ (*le vecteur query*) à $\mathbf{x}^{<m>}$ (*le vecteur key*) afin de déterminer l'importance que $\mathbf{x}^{<t>}$ accorde à $\mathbf{x}^{<m>}$ dans le mécanisme d'attention. On utilise pour cela le produit scalaire $\mathbf{x}^{<t>} \text{Tr } \mathbf{x}^{<m>}$ qui est une mesure de similarité entre les deux vecteurs.

Pour normaliser, on applique la fonction Softmax :

$$a_{tm} = \frac{\exp(\mathbf{x}^{<t>} \text{Tr } \mathbf{x}^{<m>})}{\sum_{k=1}^T \exp(\mathbf{x}^{<t>} \text{Tr } \mathbf{x}^{<k>})}$$

De plus, pour augmenter la capacité du modèle, on utilise des matrices entraînaables (chacune est une couche de réseau neuronal à propagation avant) pour transformer les vecteurs d'entrée en vecteurs *query*, *key* et *value* :

$$\text{Value} = \mathbf{X} \mathbf{W}_V$$

$$\text{Query} = \mathbf{X} \mathbf{W}_Q$$

$$\text{KEY} = \mathbf{X} \mathbf{W}_K$$

$$\mathbf{Y} = \text{Softmax}[\text{Query}(\text{Key})^{\text{Tr}}] \text{Value}$$

Où La fonction Softmax est appliquée séparément à chaque ligne (à chaque pas temporel).

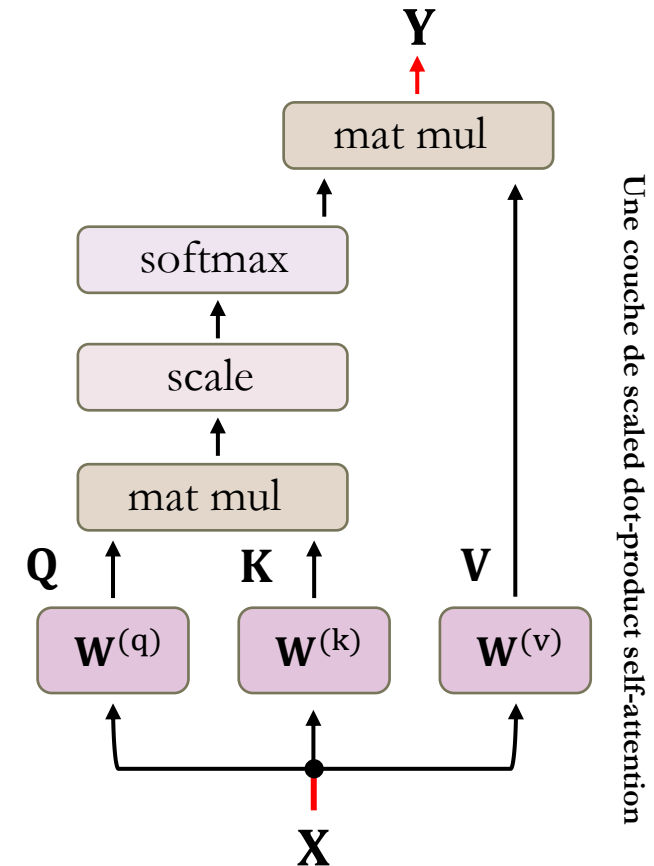
Transformers

self-attention & scaled self-attention

En résumé, l'**auto-attention (Self-Attention)** est un mécanisme qui permet à un modèle de regarder l'ensemble des autres tokens dans une séquence lorsqu'il traite un token donné.

Pour éviter que $\mathbf{QK}^{\text{Tr.}}$ ne devienne trop grand, nous pouvons re-scaler le produit des vecteurs Query et Key (avec racine carrée de la dimension $\mathbf{Q}$ ou $\mathbf{K}$) avant d'appliquer la fonction Softmax.

$$\mathbf{Y} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \equiv \text{Softmax} \left[\frac{\mathbf{QK}^{\text{Tr.}}}{\sqrt{d_k}} \right] \mathbf{V}$$

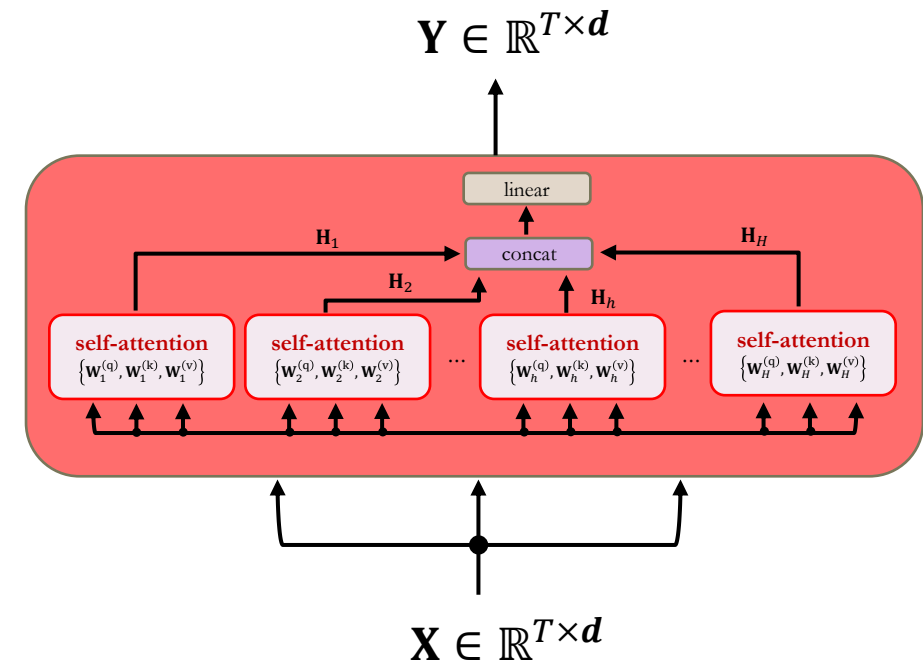


Transformers

Multi-head self-attention

Nous pouvons considérer ce que nous avons déjà vu comme une **tête d'attention**. Dans les tâches complexes, une seule tête d'attention peut ne pas apprendre tous les motifs. Nous pouvons donc utiliser plusieurs têtes d'attention en parallèle.

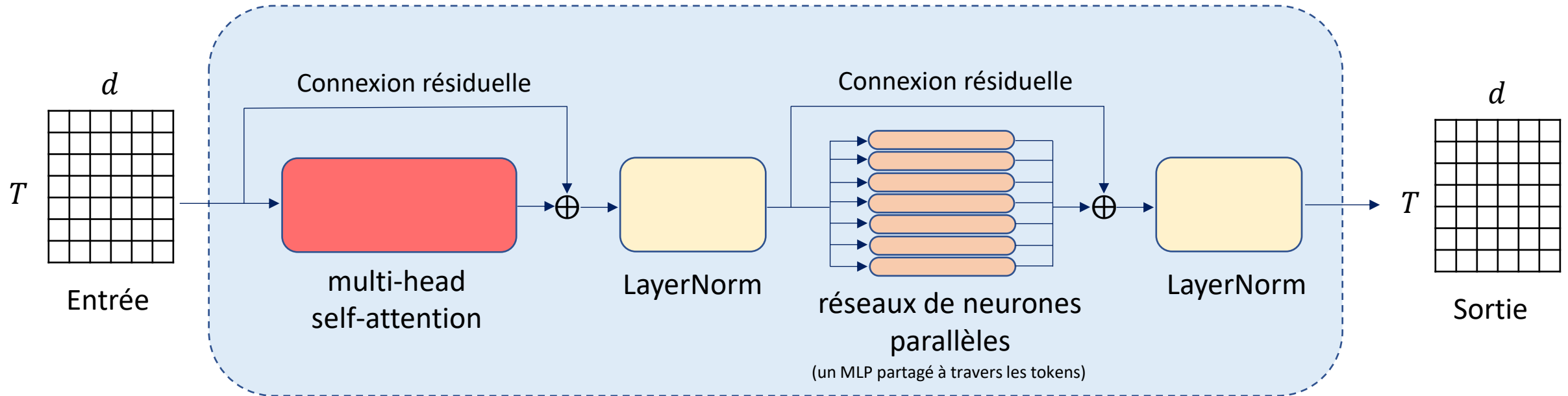
Nous aurons donc plusieurs modules d'auto-attention avec leur propre matrice. Nous concaténons ensuite les sorties et utilisons une couche linéaire (feed-forward) pour les combiner.



Approfondir les Transformers

ajouter des couches pour plus de puissance

Une couche de Deep Transformer



- Nous créons des Transformers profonds en empilant plusieurs couches de Transformers, et chaque couche contient un bloc d'auto-attention multi-têtes.
- Les connexions résiduelles, aussi appelées connexions sautées, facilitent l'apprentissage des réseaux neuronaux profonds en permettant à l'information de contourner certaines couches pour atteindre directement les couches plus profondes. Cela aide à atténuer le problème du gradient qui disparaît et améliore la capacité du réseau à apprendre et à généraliser.
- On utilise la normalisation des couches (LayerNorm) pour rendre l'apprentissage plus stable et plus rapide. Cette technique ajuste les valeurs d'activation d'un échantillon en normalisant ses caractéristiques (ou "features") individuellement, sans dépendre des autres exemples du lot.
- Un **MLP** (*Feed-forward NN*) est appliqué indépendamment sur chaque position temporelle, afin d'extraire des caractéristiques plus complexes et pertinentes.

Transformers

Encodage positionnel (Positional Encoding)

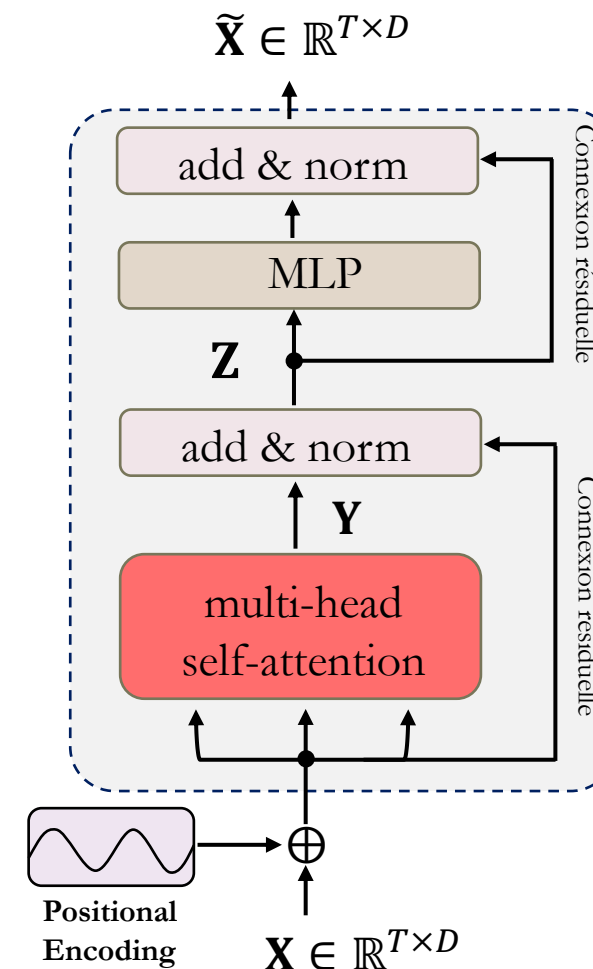
Lors de l'analyse de données séquentielles, l'ordre des « tokens » est crucial. Par exemple, l'ordre dans lequel les variations de tension apparaissent dans un ECG déterminera la présence d'une arythmie ou d'un rythme cardiaque normal.

L'architecture de base des transformateurs dont nous avons parlé ne prend pas intrinsèquement en compte l'ordre des tokens. Pour leur permettre d'intégrer l'ordre des tokens dans leur traitement, nous utilisons le codage positionnel.

Le codage positionnel ajoute des informations sur la position de chaque token dans la séquence. **Cela permet au modèle de distinguer les séquences qui contiennent les mêmes valeurs mais dans un ordre différent.**

$$\tilde{\mathbf{X}} = \text{LayerNorm}[\text{MLP}[\mathbf{Z}] + \mathbf{Z}]$$

$$\mathbf{Z} = \text{LayerNorm}[\mathbf{Y} + \mathbf{X}]$$



Transformers

Encodage positionnel (Positional Encoding)

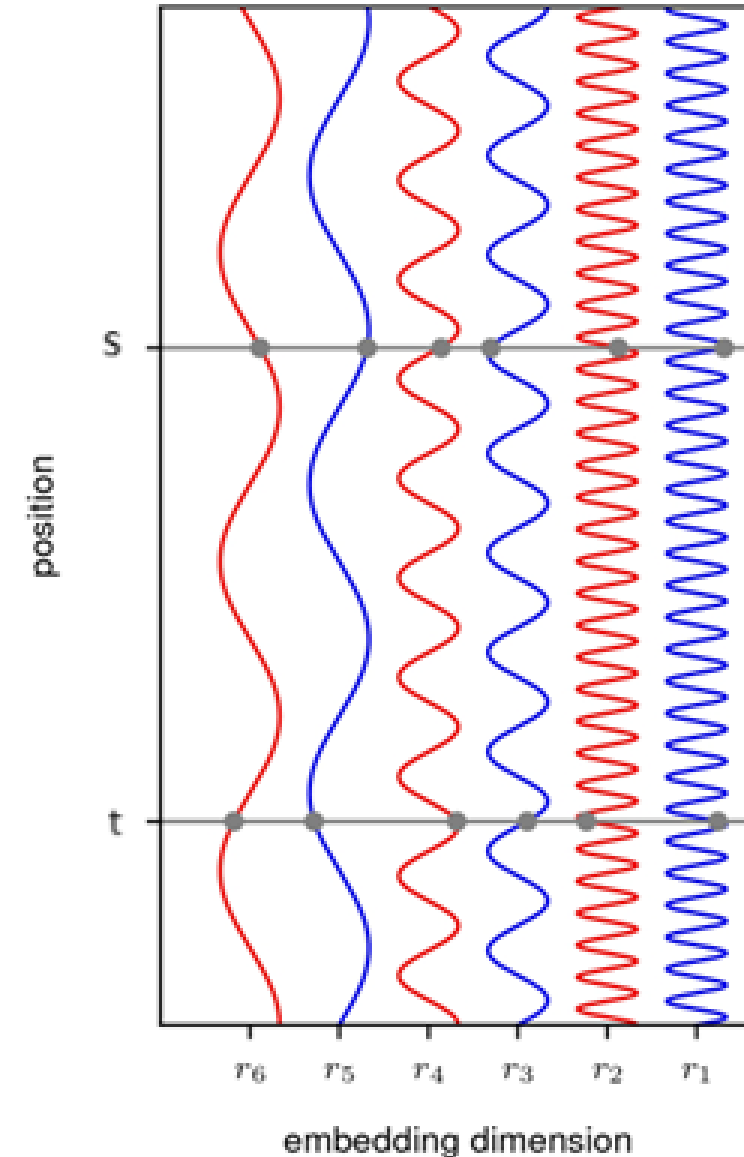
En considérant un ensemble de tokens $\mathbf{X} \in \mathbb{R}^{T \times D}$, soit $\{\mathbf{x}^{<1>}, \dots, \mathbf{x}^{<t>}, \dots, \mathbf{x}^{<T>}\}$, on associe à chaque position t un vecteur d'encodage positionnel $\mathbf{r}^{<t>}$, que l'on combine ensuite avec l'embedding du token d'entrée $\mathbf{x}^{<t>}$.

Une approche intuitive serait de concaténer $\mathbf{r}^{<t>}$ et $\mathbf{x}^{<t>}$, mais cela augmenterait la dimension. À la place, on préfère généralement additionner les deux vecteurs.

Une méthode couramment utilisée est l'encodage sinusoïdal. Le vecteur d'encodage positionnel $\mathbf{r}^{<t>} \in \mathbb{R}^d$ est défini comme suit :

$$\mathbf{r}_i^{<t>} = \begin{cases} \sin\left(\frac{t}{L^{i/d}}\right), & \text{si } i \text{ est pair} \\ \cos\left(\frac{t}{L^{(i-1)/d}}\right), & \text{si } i \text{ est impair} \end{cases}$$

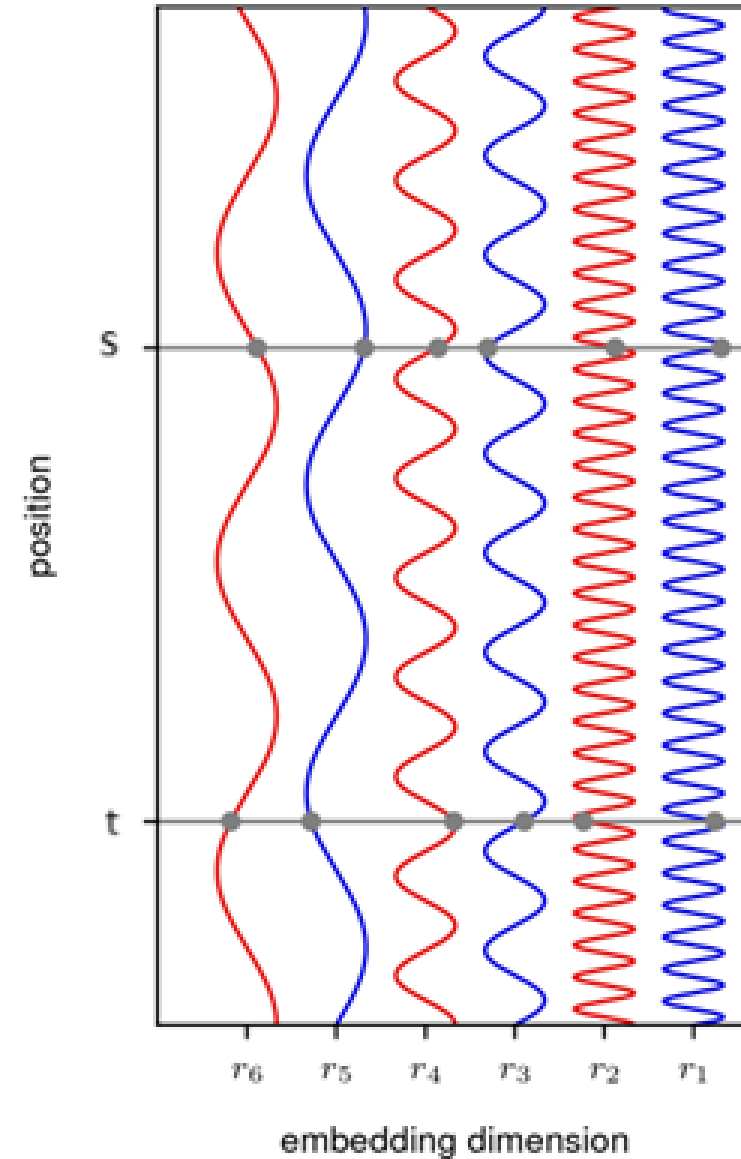
où i (de 0 à $d - 1$) est l'indice de dimension. L est une constante utilisée pour étaler les fréquences des fonctions sinusoïdales dans les différentes dimensions (par exemple fixé à 10 000).



Exercice 3 – Encodage positionnel (Positional Encoding)

On suppose que $d = 4$ (dimension de l'espace d'embedding) et que $L = 10\,000$. Écrivez les composantes du vecteur d'encodage positionnel $\mathbf{r}^{<t>}$ pour une position t donnée.

- 1- Calculez les exposants.
- 2- Simplifiez les expressions.
- 3- Expliquez pourquoi les fréquences varient selon la dimension.



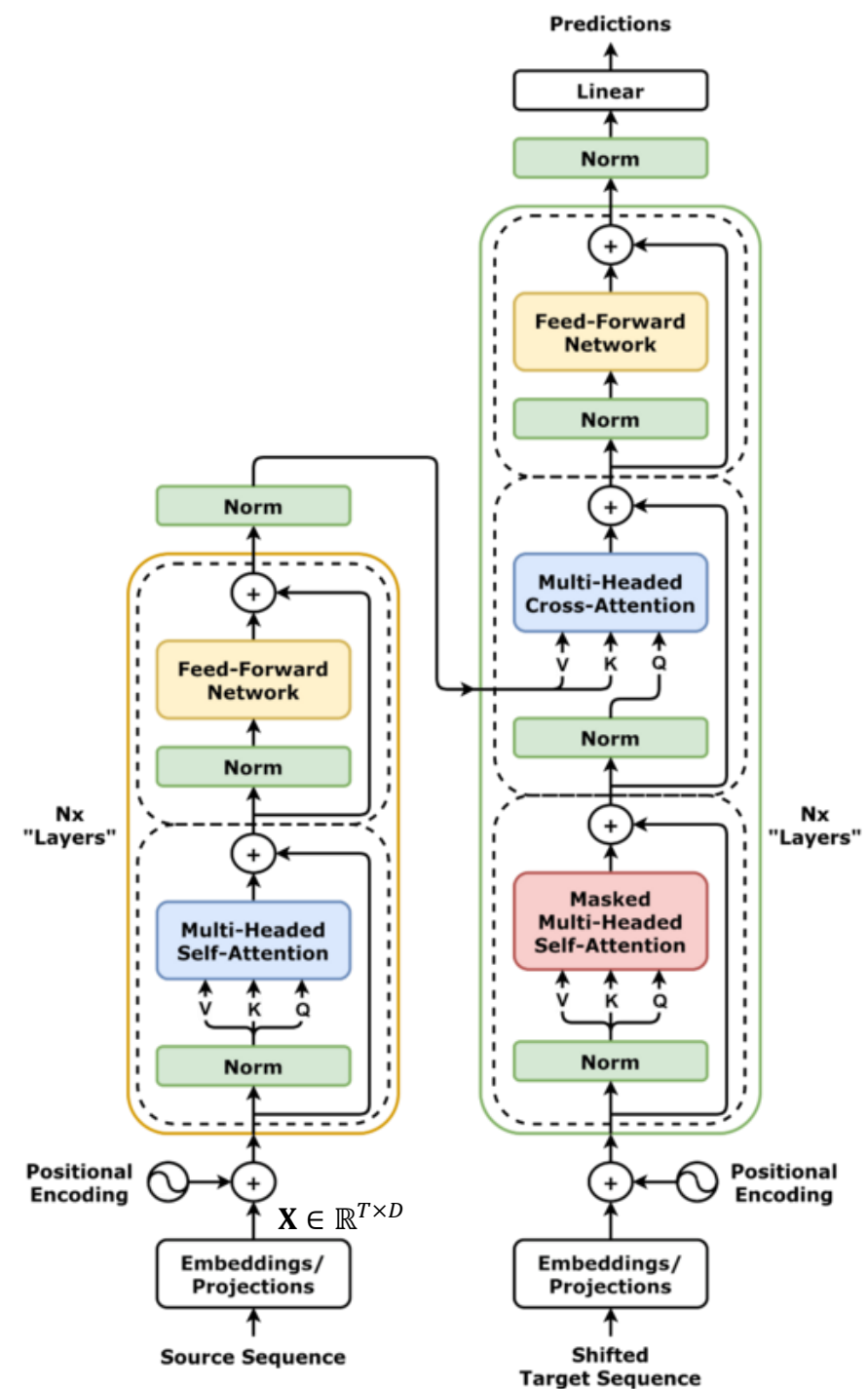
Transformers

Ce que nous avons vu est le décodeur du transformer. Dans de nombreuses applications de traitement du signal, on utilise uniquement l'**encodeur** du Transformer pour extraire des caractéristiques, puis un classificateur ou un régresseur pour produire une prédiction.

Cependant, l'architecture complète du Transformer inclut aussi un **décodeur**, utilisé principalement pour **générer des séquences** (par exemple : du texte, des signaux, etc.).

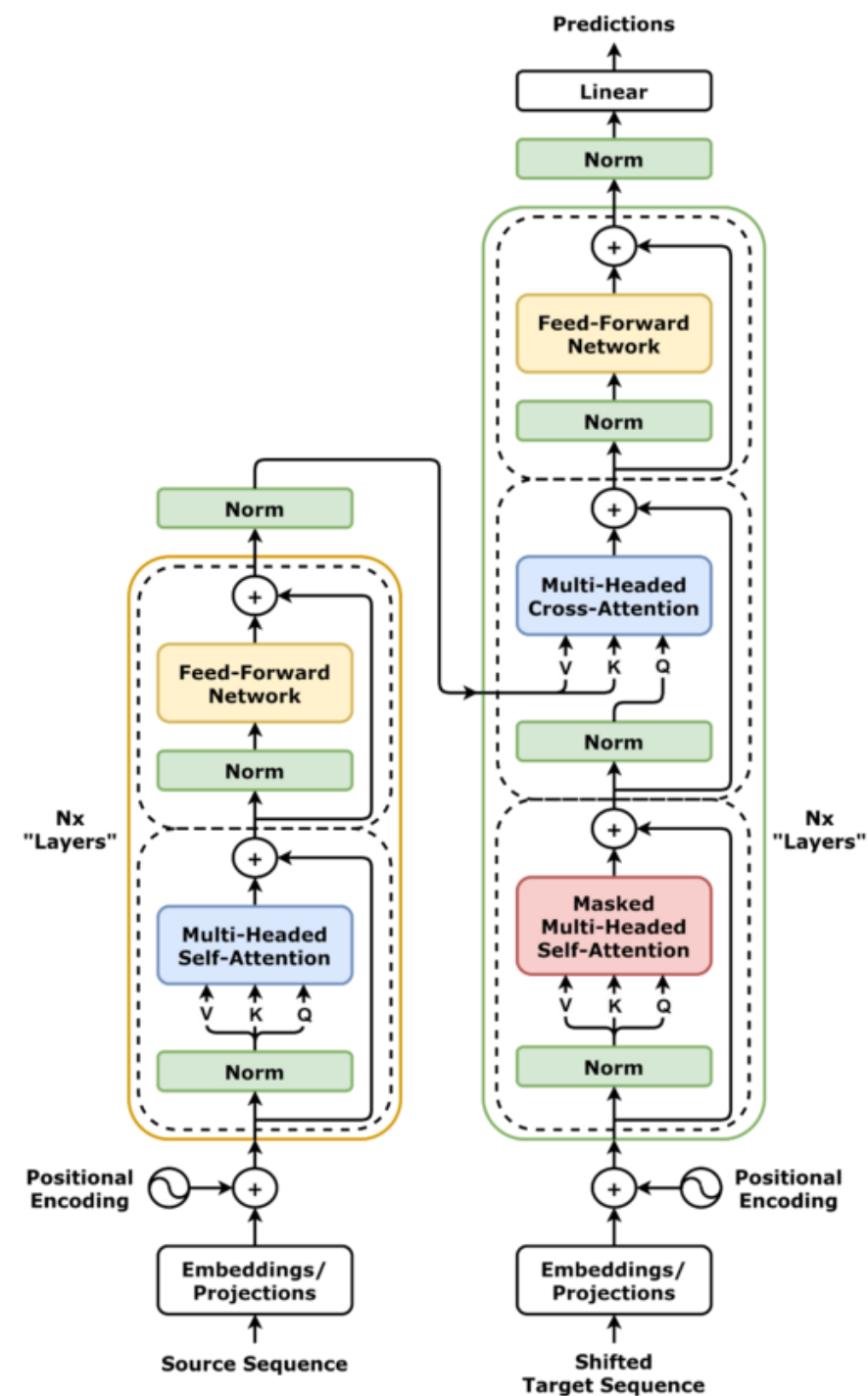
Le **décodeur** est similaire à l'encodeur, avec quelques différences importantes :

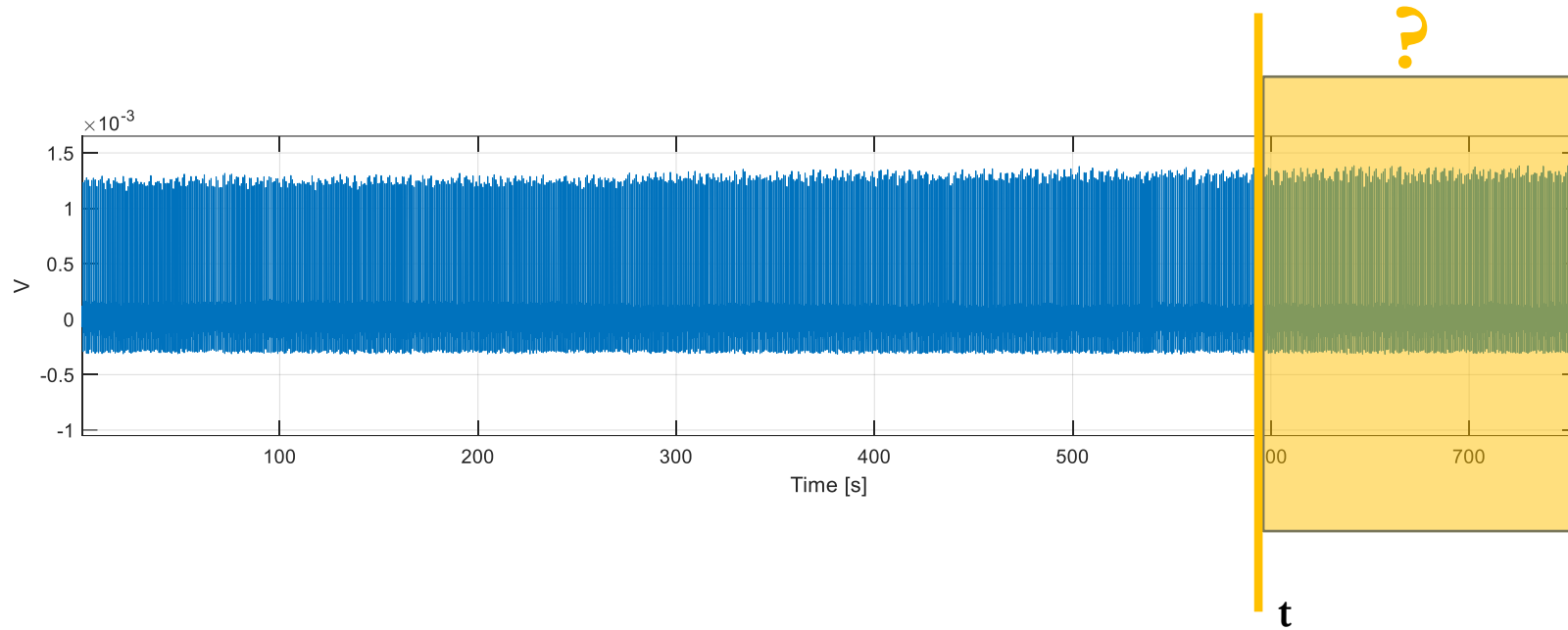
1. Il reçoit une **version décalée de la séquence cible** : à chaque étape, le modèle ne voit que les **sorties déjà produites**, jamais les futures. Cela permet une génération pas à pas. Ce mécanisme s'appelle aussi **self-attention masquée** (*masked self-attention*).
2. Il ajoute une **cross-attention** : un mécanisme similaire à la self-attention, mais où les **clés et valeurs viennent de l'encodeur**, et les requêtes du décodeur. Cela permet au décodeur de s'appuyer sur les informations encodées.



Exemple d'application Transformer : génération à partir d'ECG

1. **Encodeur** : Analyse une séquence d'ECG (électrocardiogramme) pour extraire des caractéristiques importantes du signal, capturant les dépendances temporelles complexes.
2. **Décodeur** : Utilise ces caractéristiques pour générer une séquence prédictive, par exemple :
 1. Prédire la prochaine portion du signal ECG (reconstruction ou prévision),
 2. Générer des signaux synthétiques pour l'augmentation des données,
 3. Traduire l'ECG en une séquence d'annotations cliniques (ex. détection d'arythmies).





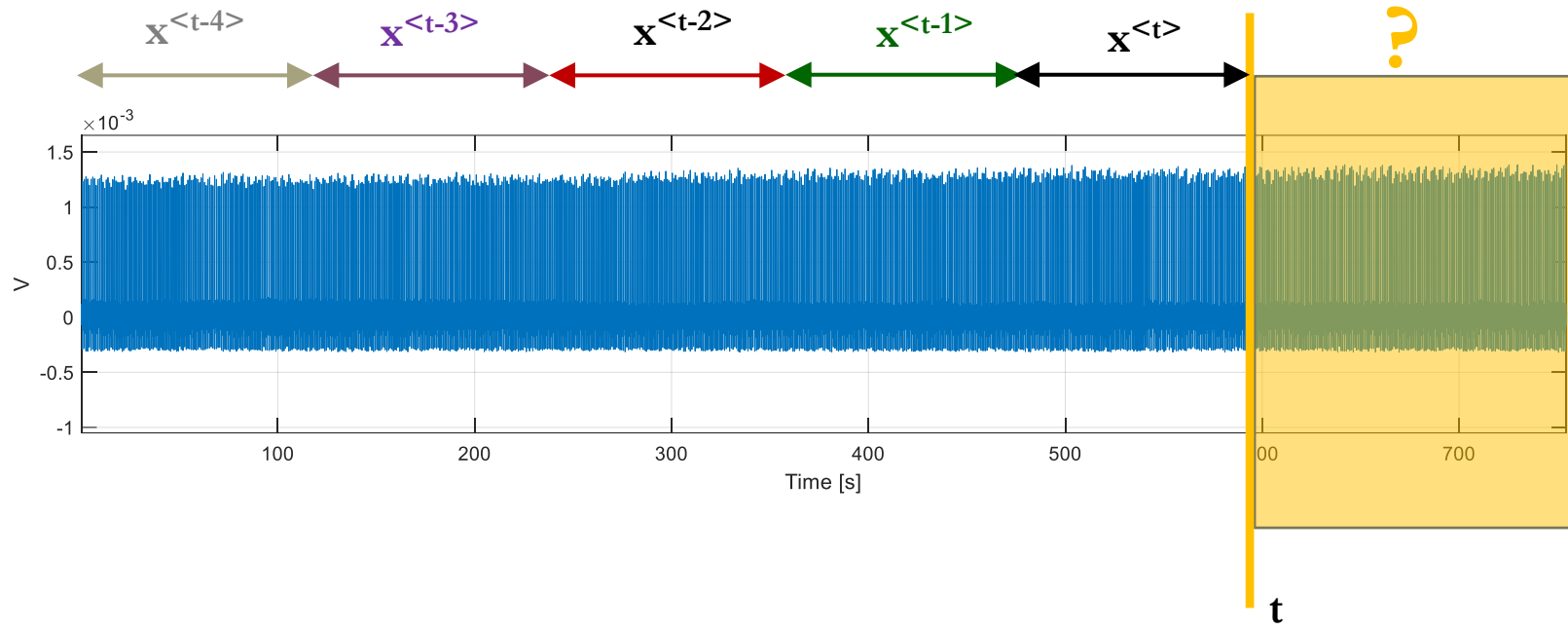
Concevoir une **LSTM (RNN)** pour la tâche :

- Choisir le type de RNN adapté à la tâche
- Définir le vecteur d'entrée au pas de temps t (quel est-il et quelle est sa dimension possible)
- Définir le nombre de pas de temps
- Définir la cible du réseau

Exercice 4

Prédiction de l'apparition de l'apnée du sommeil dans le futur à partir de l'ECG

Prenons l'exemple de la prévision de l'apparition d'un épisode d'apnée du sommeil dans les 2 minutes à venir, en analysant les 10 dernières minutes du signal ECG enregistré.



Exercise 3